The Concise TypeScript Book

Simone Poggiali

The Concise TypeScript Book

The Concise TypeScript Book bietet einen umfassenden und prägnanten Überblick über die Möglichkeiten von TypeScript. Es enthält klare Erklärungen zu allen Aspekten der neuesten Sprachversion, vom leistungsstarken Typsystem bis hin zu fortgeschrittenen Funktionen.

Ganz gleich, ob Sie Anfänger oder erfahrener Entwickler sind: Dieses Buch ist eine wertvolle Ressource, um Ihr Verständnis und Ihre Kenntnisse in TypeScript zu vertiefen.

Dieses Buch ist vollständig kostenlos und quelloffen.

Ich bin der Überzeugung, dass hochwertige technische Bildung für alle zugänglich sein sollte. Aus diesem Grund stelle ich das Buch kostenlos zur Verfügung und aktualisiere es regelmäßig mit Verbesserungen und neuen Beispielen.

Entdecken Sie **The Concise TypeScript Book Plus Edition**.



Für Leser, die über die Open-Source-Ausgabe hinausgehen möchten, enthält **The Concise TypeScript Book Plus Edition: React and Real-World Patterns for TypeScript 7** zusätzliche und exklusive Inhalte mit Schwerpunkt auf der praktischen Anwendung.

Die Plus Edition umfasst:

- **Aktualisiert für TypeScript 7** — Erläuterungen zu den neuesten Funktionen und Sprachverbesserungen von TypeScript 7.
- **TypeScript mit React** — praktische Anleitungen zur Typisierung von Komponenten, Props, Hooks, Events,

Children, Refs und gängigen React-Mustern.
- **TypeScript-Rezepte für reale Projekte** — gezielte Beispiele für praktische Probleme, mit denen Entwickler beim Erstellen und Warten von TypeScript-Anwendungen konfrontiert sind.

Mit dem Kauf der Plus Edition unterstützen Sie außerdem direkt die Weiterentwicklung und Pflege des kostenlosen Open-Source-Buchs.

Die Plus Edition ist weltweit auf Amazon in englischer und italienischer Sprache erhältlich. Entdecken Sie die Plus Edition und kaufen Sie sie auf Amazon.

Unterstützen Sie das Projekt

Wenn Ihnen das kostenlose Buch dabei geholfen hat, einen Fehler zu beheben, ein schwieriges Konzept zu verstehen oder beruflich voranzukommen, können Sie meine Arbeit gerne mit einem frei wählbaren Betrag — empfohlen sind **$5** — oder mit einem Kaffee unterstützen.

Ihre Unterstützung hilft mir, die Inhalte aktuell zu halten und sie um neue Beispiele, klarere Erklärungen und zusätzliche praktische Anleitungen zu erweitern.





Übersetzungen

Dieses Buch wurde in mehrere Sprachen übersetzt, darunter:

Englisch

Bulgarisch

Deutsch

Französisch

Indonesisch

Italienisch

Japanisch

Koreanisch

Polnisch

Portugiesisch (Brasilien)

Schwedisch

Türkisch

Vietnamese

Chinesisch

Spanisch

Thailändisch

Russisch

Arabisch Tschechisch

Downloads und Website

Sie können außerdem die EPUB-Version herunterladen:

https://github.com/gibbok/typescript-book/tree/main/downloads

Eine Online-Version ist verfügbar unter:

https://gibbok.github.io/typescript-book

Inhaltsverzeichnis

- The Concise TypeScript Book
  - Unterstützen Sie das Projekt
  - Übersetzungen
  - Downloads und Website
  - Inhaltsverzeichnis
  - Einführung
  - Über den Autor
  - Einführung in TypeScript
    - Was ist TypeScript?
    - Warum TypeScript?
    - TypeScript und JavaScript
    - TypeScript-Codegenerierung
    - Modernes JavaScript jetzt verwenden (Downleveling)
  - Erste Schritte mit TypeScript
    - Installation
    - Konfiguration
    - TypeScript-Konfigurationsdatei
      - target
      - lib

        - strict
        - module
        - moduleResolution
        - esModuleInterop
        - jsx
        - skipLibCheck
        - files
        - include
        - exclude
    - importHelpers
    - Empfehlungen für die Migration zu TypeScript
- Das Typsystem erkunden
    - Der TypeScript-Sprachdienst
    - Strukturelle Typisierung
    - Grundlegende Vergleichsregeln von TypeScript
    - Typen als Mengen
    - Einen Typ zuweisen: Typdeklarationen und Typassertionen
        - Typdeklaration
        - Typassertion
        - Umgebungsdeklarationen
    - Eigenschaftsprüfung und Prüfung auf überschüssige Eigenschaften
    - Schwache Typen
    - Strikte Prüfung von Objektliteralen (Freshness)
    - Typinferenz
    - Fortgeschrittenere Typinferenz
    - Typerweiterung
    - Const
        - Const-Modifikator für Typparameter
        - Const-Assertion
    - Explizite Typannotation

        - Type Narrowing
            - Bedingungen
            - Auslösen eines Fehlers oder Zurückkehren
            - Diskriminierte Union
            - Benutzerdefinierte Type Guards
            - Switch-true Narrowing
    - Primitive Typen
        - string
        - boolean
        - number
        - bigint
        - Symbol
        - null und undefined
        - Array
        - any
    - Typannotationen
    - Optionale Eigenschaften
    - Schreibgeschützte Eigenschaften
    - Indexsignaturen
    - Erweitern von Typen
    - Literaltypen
    - Literaltypinferenz
    - strictNullChecks
    - Enums
        - Numerische Enums
        - String-Enums
        - Konstante Enums
        - Reverse Mapping
        - Ambient Enums
        - Berechnete und konstante Member
    - Narrowing
        - typeof Type Guards

        - Truthiness-Narrowing
        - Equality-Narrowing
        - Narrowing mit dem in-Operator
        - Narrowing mit instanceof
    - Zuweisungen
    - Kontrollflussanalyse
    - Typprädikate
    - Diskriminierte Unions
    - Der Typ never
    - Vollständigkeitsprüfung
    - Objekttypen
    - Tupeltyp (anonym)
    - Benannter Tupeltyp (beschriftet)
    - Tupel mit fester Länge
    - Union-Typ
    - Schnittmengentypen
    - Typindizierung
    - Typ aus einem Wert
    - Typ aus dem Funktionsrückgabewert
    - Typ aus einem Modul
    - Gemappte Typen
    - Modifizierer gemappter Typen
    - Bedingte Typen
    - Distributive bedingte Typen
    - Typinferenz mit infer in bedingten Typen
    - Vordefinierte bedingte Typen
    - Template-Union-Typen
    - Typ any
    - Typ unknown
    - Typ void
    - Typ never
    - Interface und Typ

        - Allgemeine Syntax
        - Grundlegende Typen
        - Objekte und Interfaces
        - Union- und Intersection-Typen
    - Eingebaute primitive Typen
    - Häufig verwendete eingebaute JS-Objekte
    - Überladungen
    - Zusammenführung und Erweiterung
    - Unterschiede zwischen Typ und Interface
    - Klasse
        - Allgemeine Syntax einer Klasse
        - Konstruktor
        - Private und geschützte Konstruktoren
        - Zugriffsmodifikatoren
        - Get und Set
        - Auto-Accessors in Klassen
        - this
        - Parametereigenschaften
        - Abstrakte Klassen
        - Mit Generics
        - Decorators
            - Klassen-Decorators
            - Eigenschafts-Decorator
            - Methoden-Decorator
            - Getter- und Setter-Decorators
            - Decorator-Metadaten
        - Vererbung
        - Statische Member
        - Initialisierung von Eigenschaften
        - Methodenüberladung
    - Generics
        - Generischer Typ

        - Generische Klassen
        - Einschränkungen für Generics
        - Kontextbezogenes Narrowing für Generics
    - Strukturelle Typen mit Typlöschung
    - Namespaces
    - Symbole
    - Triple-Slash-Direktiven
    - Typmanipulation
        - Erstellen von Typen aus Typen
        - Indexed Access Types
        - Utility Types
            - Awaited<T>
            - Partial<T>
            - Required<T>
            - Readonly<T>
            - Record<K, T>
            - Pick<T, K>
            - Omit<T, K>
            - Exclude<T, U>
            - Extract<T, U>
            - NonNullable<T>
            - Parameters<T>
            - ConstructorParameters<T>
            - ReturnType<T>
            - InstanceType<T>
            - ThisParameterType<T>
            - OmitThisParameter<T>
            - ThisType<T>
            - Uppercase<T>
            - Lowercase<T>
            - Capitalize<T>
            - Uncapitalize<T>

        - NoInfer<T>
- Sonstiges
    - Fehler- und Ausnahmebehandlung
    - Mixin-Klassen
    - Asynchrone Sprachfunktionen
    - Iteratoren und Generatoren
    - TsDocs-JSDoc-Referenz
    - @types
    - JSX
    - ES6-Module
    - ES7-Potenzierungsoperator
    - Die for-await-of-Anweisung
    - Neue target-Metaeigenschaft
    - Dynamische Importausdrücke
    - “tsc –watch”
    - Non-Null-Assertion-Operator
    - Deklarationen mit Standardwerten
    - Optional Chaining
    - Nullish-Coalescing-Operator
    - Template Literal Types
    - Funktionsüberladung
    - Rekursive Typen
    - Rekursive Conditional Types
    - Unterstützung von ECMAScript-Modulen in Node
    - Assertion Functions
    - Variadische Tupeltypen
    - Boxed Types
    - Kovarianz und Kontravarianz in TypeScript
        - Optionale Varianzannotationen für Typparameter
    - Template-String-Pattern-Indexsignaturen
    - Der satisfies-Operator

- Type-Only Imports und Export
- using-Deklaration und Explicit Resource Management
  - await using-Deklaration
- Importattribute
- Syntaxprüfung regulärer Ausdrücke
- import defer

Einführung

Willkommen bei The Concise TypeScript Book! Dieser Leitfaden vermittelt Ihnen grundlegendes Wissen und praktische Fähigkeiten für eine effektive TypeScript-Entwicklung. Entdecken Sie wichtige Konzepte und Techniken zum Schreiben von sauberem, robustem Code. Ganz gleich, ob Sie Anfänger oder erfahrener Entwickler sind: Dieses Buch dient sowohl als umfassender Leitfaden als auch als praktische Referenz, um die Stärken von TypeScript in Ihren Projekten zu nutzen.

Dieses Buch behandelt TypeScript 7.0.

Über den Autor

Simone Poggiali ist ein erfahrener Staff Engineer, der seit den 90er-Jahren mit Leidenschaft professionellen Code schreibt. Im Laufe seiner internationalen Karriere hat er an zahlreichen Projekten für ein breites Kundenspektrum mitgewirkt, von Start-ups bis hin zu großen Unternehmen. Namhafte Unternehmen wie HelloFresh, Siemens, O2, Leroy Merlin und Snowplow haben von seiner Expertise und seinem Engagement profitiert.

Sie erreichen Simone Poggiali über die folgenden Plattformen:

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social
- E-Mail: gibbok.coding gmail.com

Vollständige Liste der Mitwirkenden: https://github.com/gibbok/typescript-book/graphs/contributors

Einführung in TypeScript

Was ist TypeScript?

TypeScript ist eine stark typisierte Programmiersprache, die auf JavaScript aufbaut. Sie wurde ursprünglich 2012 von Anders Hejlsberg entworfen und wird derzeit von Microsoft als Open-Source-Projekt entwickelt und gepflegt.

TypeScript wird zu JavaScript kompiliert und kann in jeder JavaScript-Laufzeitumgebung ausgeführt werden (z. B. in einem Browser oder mit Node.js auf einem Server).

TypeScript unterstützt mehrere Programmierparadigmen wie funktionale, generische, imperative und objektorientierte Programmierung und ist eine kompilierte (transpilierte) Sprache, die vor der Ausführung in JavaScript umgewandelt wird.

Warum TypeScript?

TypeScript ist eine stark typisierte Sprache, die dabei hilft, häufige Programmierfehler zu verhindern und bestimmte

Arten von Laufzeitfehlern zu vermeiden, bevor das Programm ausgeführt wird.

Eine stark typisierte Sprache ermöglicht es Entwicklern, verschiedene Einschränkungen und Verhaltensweisen des Programms in den Datentypdefinitionen festzulegen. Dadurch lässt sich die Korrektheit der Software leichter überprüfen und Fehlern vorbeugen. Dies ist besonders bei umfangreichen Anwendungen von großem Wert.

Einige Vorteile von TypeScript:

- Statische Typisierung, optional stark typisiert
- Typinferenz
- Zugriff auf Funktionen von ES6 und ES7
- Plattform- und browserübergreifende Kompatibilität
- Werkzeugunterstützung mit IntelliSense

TypeScript und JavaScript

TypeScript wird in `.ts`- oder `.tsx`-Dateien geschrieben, während JavaScript-Dateien in `.js` oder `.jsx` geschrieben werden.

Dateien mit der Erweiterung `.tsx` oder `.jsx` können die JavaScript-Syntaxerweiterung JSX enthalten, die in React für die UI-Entwicklung verwendet wird.

TypeScript ist hinsichtlich der Syntax eine typisierte Obermenge von JavaScript (ECMAScript 2015). Jeder JavaScript-Code ist gültiger TypeScript-Code, umgekehrt gilt dies jedoch nicht immer.

Betrachten Sie beispielsweise eine Funktion in einer JavaScript-Datei mit der Erweiterung `.js`, wie die folgende:

```
const sum = (a, b) => a + b;
```

Die Funktion kann durch Ändern der Dateierweiterung in `.ts` konvertiert und in TypeScript verwendet werden. Wird dieselbe Funktion jedoch mit TypeScript-Typen annotiert, kann sie ohne Kompilierung in keiner JavaScript-Laufzeitumgebung ausgeführt werden. Der folgende TypeScript-Code erzeugt einen Syntaxfehler, wenn er nicht kompiliert wird:

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript wurde entwickelt, um potenzielle Laufzeitfehler bereits zur Kompilierzeit zu erkennen, indem Entwickler ihre Absicht durch Typannotationen ausdrücken können. Dank der Typinferenz kann TypeScript außerdem bestimmte Probleme auch dann erkennen, wenn keine expliziten Typannotationen angegeben sind. Der folgende Codeausschnitt gibt beispielsweise keine TypeScript-Typen an:

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

In diesem Fall erkennt TypeScript einen Fehler und meldet:

```
Property 'y' does not exist on type '{ x: number; }'.
```

Das Typsystem von TypeScript ist weitgehend vom Laufzeitverhalten von JavaScript geprägt. Der Additionsoperator (+), der in JavaScript entweder Zeichenketten verketten oder Zahlen addieren kann, wird in TypeScript beispielsweise auf dieselbe Weise modelliert:

```
const result = '1' + 1; // Result is of type string
```

Das TypeScript-Team hat sich bewusst dafür entschieden, ungewöhnliche Verwendungen von JavaScript als Fehler zu kennzeichnen. Betrachten Sie beispielsweise den folgenden gültigen JavaScript-Code:

```
const result = 1 + true; // In JavaScript, the result is equal to 2
```

TypeScript gibt jedoch einen Fehler aus:

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

Dieser Fehler tritt auf, weil TypeScript die Typkompatibilität strikt durchsetzt und in diesem Fall eine ungültige Operation zwischen einer Zahl und einem booleschen Wert erkennt.

TypeScript-Codegenerierung

Der TypeScript-Compiler hat zwei Hauptaufgaben: die Prüfung auf Typfehler und die Kompilierung zu JavaScript. Diese beiden Prozesse sind voneinander unabhängig. Typen wirken sich nicht auf die Ausführung des Codes in einer JavaScript-Laufzeitumgebung aus, da sie bei der Kompilierung vollständig entfernt werden. TypeScript kann selbst bei vorhandenen Typfehlern JavaScript ausgeben. Hier ist ein Beispiel für TypeScript-Code mit einem Typfehler:

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // Argument of type 'string' is not assignable to parameter of type 'number'.
```

Dennoch kann weiterhin ausführbarer JavaScript-Code erzeugt werden:

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

TypeScript-Typen können nicht zur Laufzeit geprüft werden. Zum Beispiel:

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
        // 'Dog' only refers to a type, but is being used as a
value here.
        // ...
    }
};
```

Da die Typen nach der Kompilierung entfernt werden, kann dieser Code nicht in JavaScript ausgeführt werden. Um Typen zur Laufzeit zu erkennen, müssen wir einen anderen Mechanismus verwenden. TypeScript bietet mehrere Möglichkeiten; eine gängige ist die „Tagged Union“. Zum Beispiel:

```
interface Dog {
    kind: 'dog'; // Tagged union
    bark: () => void;
}
interface Cat {
    kind: 'cat'; // Tagged union
```

```
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

Die Eigenschaft „kind“ ist ein Wert, mit dem sich Objekte in JavaScript zur Laufzeit unterscheiden lassen.

Es ist auch möglich, dass ein Wert zur Laufzeit einen anderen Typ aufweist als in der Typdeklaration angegeben. Dies kann beispielsweise vorkommen, wenn ein Entwickler einen API-Typ falsch interpretiert und entsprechend falsch annotiert hat.

TypeScript ist eine Obermenge von JavaScript, daher kann das Schlüsselwort „class“ zur Laufzeit sowohl als Typ als auch als Wert verwendet werden.

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
```

```
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

In JavaScript besitzt eine „class“ eine „prototype“-Eigenschaft. Mit dem „instanceof“-Operator lässt sich prüfen, ob die Prototype-Eigenschaft eines Konstruktors an einer Stelle in der Prototype-Kette eines Objekts vorkommt.

TypeScript hat keinen Einfluss auf die Laufzeitleistung, da alle Typen entfernt werden. TypeScript verursacht jedoch einen gewissen zusätzlichen Aufwand bei der Build-Zeit.

Modernes JavaScript jetzt verwenden (Downleveling)

TypeScript kann Code für jede seit ECMAScript 3 (1999) veröffentlichte JavaScript-Version kompilieren. Das bedeutet, dass TypeScript Code mit den neuesten JavaScript-Funktionen in ältere Versionen transpilieren kann. Dieser Vorgang wird als Downleveling bezeichnet. Dadurch lässt sich modernes JavaScript verwenden, während eine maximale Kompatibilität mit älteren Laufzeitumgebungen erhalten bleibt.

Dabei ist zu beachten, dass TypeScript bei der Transpilierung in eine ältere JavaScript-Version Code erzeugen kann, der gegenüber nativen Implementierungen einen zusätzlichen Leistungsaufwand verursacht.

Nachfolgend sind einige moderne JavaScript-Funktionen aufgeführt, die in TypeScript verwendet werden können:

- ECMAScript-Module anstelle von „define“-Callbacks im AMD-Stil oder „require“-Anweisungen von CommonJS.
- Klassen anstelle von Prototypen.
- Variablendeklarationen mit „let“ oder „const“ anstelle von „var“.
- „for-of“-Schleifen oder „.forEach“ anstelle der herkömmlichen „for“-Schleife.
- Pfeilfunktionen anstelle von Funktionsausdrücken.
- Destrukturierende Zuweisung.
- Kurzschreibweisen für Eigenschafts-/Methodennamen und berechnete Eigenschaftsnamen.
- Standardparameter für Funktionen.

Durch die Nutzung dieser modernen JavaScript-Funktionen können Entwickler ausdrucksstärkeren und prägnanteren TypeScript-Code schreiben.

Erste Schritte mit TypeScript

Installation

Visual Studio Code bietet eine ausgezeichnete Unterstützung für die TypeScript-Sprache, enthält jedoch nicht den TypeScript-Compiler. Zum Installieren des TypeScript-Compilers können Sie einen Paketmanager wie npm oder yarn verwenden:

```
npm install typescript --save-dev
```

oder

```
yarn add typescript --dev
```

Committen Sie unbedingt die erzeugte Lockdatei, damit jedes Teammitglied dieselbe TypeScript-Version verwendet.

Zum Ausführen des TypeScript-Compilers können Sie die folgenden Befehle verwenden:

```
npx tsc
```

oder

```
yarn tsc
```

Es wird empfohlen, TypeScript projektbezogen statt global zu installieren, da dies einen vorhersehbareren Build-Prozess ermöglicht. Für einmalige Anwendungsfälle können Sie jedoch den folgenden Befehl verwenden:

```
npx tsc
```

oder es global installieren:

```
npm install -g typescript
```

Wenn Sie Microsoft Visual Studio verwenden, können Sie TypeScript als NuGet-Paket für Ihre MSBuild-Projekte beziehen. Führen Sie in der NuGet-Paket-Manager-Konsole den folgenden Befehl aus:

```
Install-Package Microsoft.TypeScript.MSBuild
```

Bei der TypeScript-Installation werden zwei ausführbare Dateien installiert: „tsc“ als TypeScript-Compiler und „tsserver“ als eigenständiger TypeScript-Server. Der eigenständige Server enthält den Compiler und Sprachdienste, die von Editoren und IDEs für eine intelligente Codevervollständigung genutzt werden können.

Darüber hinaus stehen mehrere TypeScript-kompatible Transpiler zur Verfügung, beispielsweise Babel (über ein Plugin) oder swc. Mit diesen Transpilern lässt sich TypeScript-Code in andere Zielsprachen oder -versionen umwandeln.

TypeScript 7.0 wurde in Go als native Implementierung des Compilers und Sprachdienstes neu geschrieben. Es nutzt Multithreading mit gemeinsamem Speicher und weitere Optimierungen, um vollständige Builds und Editorfunktionen zu beschleunigen und so die Feedbackzeit während der Entwicklung zu verkürzen.

Einige Leistungsfunktionen von TypeScript 7.0 lassen sich konfigurieren. Die Typprüfung kann mit `--checkers` in parallelen Workern ausgeführt werden; mehr Worker können große Projekte beschleunigen, benötigen jedoch mehr Arbeitsspeicher. Der neu entwickelte `--watch`-Modus verbessert

die plattformübergreifende Dateiüberwachung. TypeScript 7.0 enthält noch keine Compiler-API (Stand Juli 2026). Daher können Werkzeuge, die weiterhin die API von TypeScript 6.0 benötigen, mithilfe von `@typescript/typescript6` oder npm-Aliasnamen parallel zu TypeScript 7.0 ausgeführt werden.

Konfiguration

TypeScript kann über die CLI-Optionen von tsc oder mithilfe einer eigenen Konfigurationsdatei namens tsconfig.json konfiguriert werden, die im Stammverzeichnis des Projekts abgelegt wird.

Mit dem folgenden Befehl können Sie eine tsconfig.json-Datei erzeugen, die bereits mit empfohlenen Einstellungen ausgefüllt ist:

```
tsc --init
```

Wenn Sie den Befehl `tsc` lokal ausführen, kompiliert TypeScript den Code mit der Konfiguration aus der nächstgelegenen tsconfig.json-Datei.

Hier sind einige Beispiele für CLI-Befehle, die mit den Standardeinstellungen ausgeführt werden:

```
tsc main.ts // Compile a specific file (main.ts) to JavaScript
tsc src/*.ts // Compile any .ts files under the 'src' folder to JavaScript
tsc app.ts util.ts --outfile index.js // Compile two TypeScript files (app.ts and util.ts) into a single JavaScript file (index.js)
```

TypeScript-Konfigurationsdatei

Eine tsconfig.json-Datei dient zur Konfiguration des TypeScript-Compilers (tsc). Üblicherweise wird sie zusammen mit der Datei `package.json` im Stammverzeichnis des Projekts abgelegt.

Hinweise:

- tsconfig.json akzeptiert Kommentare, obwohl sie im JSON-Format vorliegt.
- Es wird empfohlen, diese Konfigurationsdatei anstelle der Befehlszeilenoptionen zu verwenden.

Unter dem folgenden Link finden Sie die vollständige Dokumentation und das zugehörige Schema:

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

Im Folgenden finden Sie eine Liste gängiger und nützlicher Konfigurationen:

target

Mit der Eigenschaft „target“ wird festgelegt, in welche ECMAScript-Version Ihr TypeScript-Code ausgegeben bzw. kompiliert werden soll. Für moderne Browser ist ES6 eine gute Wahl. Hinweis: Die Unterstützung für ES5 wurde in TypeScript 6.0 als veraltet markiert und wird in TypeScript 7.0 nicht mehr unterstützt.

lib

Mit der Eigenschaft „lib“ wird festgelegt, welche Bibliotheksdateien zur Kompilierzeit einbezogen werden. TypeScript schließt automatisch APIs für Funktionen ein, die in der Eigenschaft „target“ angegeben sind. Für bestimmte Anforderungen können jedoch einzelne Bibliotheken ausgelassen oder gezielt ausgewählt werden. Bei einem Serverprojekt könnten Sie beispielsweise die Bibliothek „DOM“ ausschließen, da diese nur in einer Browserumgebung nützlich ist.

strict

Die Option „strict“ verbessert die Typsicherheit, indem sie strengere Prüfungen aktiviert. Ab TypeScript 6.0 ist sie standardmäßig aktiviert; andernfalls sollten Sie sie in Ihrer tsconfig.json explizit auf true setzen. Durch die Aktivierung von „strict“ kann TypeScript:

- für jede Quelldatei Code mit „use strict“ ausgeben.
- „null“ und „undefined“ bei der Typprüfung berücksichtigen.
- die Verwendung des Typs „any“ deaktivieren, wenn keine Typannotationen vorhanden sind.
- bei Verwendung des Ausdrucks „this“ einen Fehler ausgeben, wenn dieser andernfalls den Typ „any“ implizieren würde.

module

Die Eigenschaft „module“ legt das Modulsystem fest, das vom kompilierten Programm unterstützt wird. Zur Laufzeit wird ein

Modullader verwendet, um Abhängigkeiten anhand des angegebenen Modulsystems zu finden und auszuführen.

Die in JavaScript am häufigsten verwendeten Modullader sind Node.js CommonJS für serverseitige Anwendungen und RequireJS für AMD-Module in browserbasierten Webanwendungen. TypeScript kann Code für verschiedene Modulsysteme ausgeben, darunter UMD, System, ESNext, ES2015/ES6 und ES2020. Das Modulsystem sollte entsprechend der Zielumgebung und dem darin verfügbaren Mechanismus zum Laden von Modulen gewählt werden.

Hinweis: Die Unterstützung für ältere Modulsysteme (AMD, UMD, SystemJS) wurde in TypeScript 6.0 als veraltet markiert und ist in TypeScript 7.0 nicht mehr verfügbar.

moduleResolution

Die Eigenschaft „moduleResolution“ legt die Strategie zur Modulauflösung fest. Verwenden Sie für modernen TypeScript-Code „nodenext“ oder „bundler“. Die Strategie „classic“ wird nur bei alten TypeScript-Versionen (vor 1.6) eingesetzt.

esModuleInterop

Die Eigenschaft „esModuleInterop“ ermöglicht Standardimporte aus CommonJS-Modulen, die nicht über die Eigenschaft „default“ exportiert haben. Sie stellt einen Shim bereit, um die Kompatibilität im ausgegebenen JavaScript sicherzustellen. Nach der Aktivierung dieser Option können wir `import MyLibrary from "my-library"` anstelle von `import * as MyLibrary from "my-library"` verwenden.

„esModuleInterop“ musste ursprünglich explizit aktiviert werden, um Breaking Changes zu vermeiden, ist jedoch seit Langem die empfohlene Standardeinstellung. Die Deaktivierung kann bei der gemeinsamen Verwendung von CommonJS und ESM zu subtilen Laufzeitproblemen führen. Hinweis: Ab TypeScript 6.0 ist dieses sicherere Interop-Verhalten immer aktiviert.

In TypeScript 6.0 wurden einige ältere Konfigurationsoptionen und Syntaxformen als veraltet markiert oder über altes Verhalten umgestellt. In TypeScript 7.0 führen sie zu nicht behebbaren Fehlern oder zeigen keine Wirkung.

Die veralteten Funktionen, die zu nicht behebbaren Fehlern ohne Wirkung geworden sind, lauten:

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`
- Deaktivieren von `esModuleInterop` oder `allowSyntheticDefaultImports`
- Deaktivieren von `alwaysStrict`
- Schlüsselwort `module` in Namespace-Deklarationen
- `asserts` bei Importen
- `/// <reference no-default-lib />` unter `skipDefaultLibCheck`
- CLI-Dateipfade mit einer lokalen `tsconfig.json`, sofern nicht `--ignoreConfig` verwendet wird

jsx

Die Eigenschaft „jsx“ gilt nur für in ReactJS verwendete .tsx-Dateien und steuert, wie JSX-Konstrukte in JavaScript kompiliert werden. Eine gängige Option ist „preserve“. Dabei wird in eine .jsx-Datei kompiliert und das JSX unverändert beibehalten, sodass es für weitere Transformationen an andere Werkzeuge wie Babel übergeben werden kann.

skipLibCheck

Die Eigenschaft „skipLibCheck“ verhindert, dass TypeScript sämtliche importierten Drittanbieterpakete einer Typprüfung unterzieht. Dadurch verkürzt sich die Kompilierzeit eines Projekts. TypeScript prüft Ihren Code weiterhin anhand der von diesen Paketen bereitgestellten Typdefinitionen.

files

Die Eigenschaft „files“ gibt dem Compiler eine Liste von Dateien an, die stets in das Programm einbezogen werden müssen.

include

Die Eigenschaft „include“ gibt dem Compiler eine Liste der Dateien an, die einbezogen werden sollen. Diese Eigenschaft erlaubt Glob-ähnliche Muster, etwa „*“ *für jedes Unterverzeichnis,* „“ für jeden Dateinamen und „?“ für optionale Zeichen.

exclude

Die Eigenschaft „exclude“ gibt dem Compiler eine Liste der Dateien an, die nicht in die Kompilierung einbezogen werden

sollen. Dazu können Dateien wie „node_modules“ oder Testdateien gehören. Hinweis: tsconfig.json erlaubt Kommentare.

importHelpers

TypeScript verwendet Hilfscode, wenn es Code für bestimmte fortgeschrittene oder heruntergestufte JavaScript-Funktionen erzeugt. Standardmäßig werden diese Hilfsfunktionen in den Dateien dupliziert, die sie verwenden. Die Option `importHelpers` importiert diese Hilfsfunktionen stattdessen aus dem Modul `tslib`, wodurch die JavaScript-Ausgabe effizienter wird.

Empfehlungen für die Migration zu TypeScript

Bei großen Projekten empfiehlt sich ein schrittweiser Übergang, bei dem TypeScript- und JavaScript-Code zunächst nebeneinander bestehen. Nur kleine Projekte können in einem Schritt zu TypeScript migriert werden.

Der erste Schritt dieses Übergangs besteht darin, TypeScript in den Build-Prozess einzuführen. Dies kann über die Compileroption „allowJs“ erfolgen, die es .ts- und .tsx-Dateien ermöglicht, neben vorhandenen JavaScript-Dateien zu bestehen. Da TypeScript bei einer Variablen auf den Typ „any“ zurückfällt, wenn es den Typ nicht aus JavaScript-Dateien ableiten kann, empfiehlt es sich, „noImplicitAny“ zu Beginn der Migration in den Compileroptionen zu deaktivieren.

Im zweiten Schritt stellen Sie sicher, dass Ihre JavaScript-Tests zusammen mit TypeScript-Dateien funktionieren, damit Sie während der Konvertierung jedes Moduls Tests ausführen können. Wenn Sie Jest verwenden, sollten Sie `ts-jest` in

Betracht ziehen, das Tests von TypeScript-Projekten mit Jest ermöglicht.

Im dritten Schritt nehmen Sie Typdeklarationen für Drittanbieterbibliotheken in Ihr Projekt auf. Diese Deklarationen sind entweder im Paket enthalten oder auf DefinitelyTyped zu finden. Sie können unter https://www.typescriptlang.org/dt/search danach suchen und sie wie folgt installieren:

```
npm install --save-dev @types/package-name
```

oder

```
yarn add --dev @types/package-name
```

Im vierten Schritt migrieren Sie Modul für Modul nach einem Bottom-up-Ansatz. Folgen Sie dabei Ihrem Abhängigkeitsgraphen und beginnen Sie bei den Blättern. Die Idee besteht darin, zunächst Module zu konvertieren, die nicht von anderen Modulen abhängen. Zur Visualisierung der Abhängigkeitsgraphen können Sie das Werkzeug „madge“ verwenden.

Geeignete Module für diese ersten Konvertierungen sind Hilfsfunktionen und Code im Zusammenhang mit externen APIs oder Spezifikationen. TypeScript-Typdefinitionen können automatisch aus Swagger-Verträgen, GraphQL- oder JSON-Schemas erzeugt und in Ihr Projekt aufgenommen werden.

Wenn keine Spezifikationen oder offiziellen Schemas verfügbar sind, können Sie Typen aus Rohdaten erzeugen, etwa aus dem von einem Server zurückgegebenen JSON. Es wird jedoch

empfohlen, Typen aus Spezifikationen statt aus Daten zu erzeugen, damit keine Grenzfälle übersehen werden.

Verzichten Sie während der Migration auf Code-Refactoring und konzentrieren Sie sich ausschließlich darauf, Ihren Modulen Typen hinzuzufügen.

Im fünften Schritt aktivieren Sie „noImplicitAny“. Dadurch wird erzwungen, dass alle Typen bekannt und definiert sind, was die Arbeit mit TypeScript in Ihrem Projekt verbessert.

Während der Migration können Sie die Direktive `@ts-check` verwenden, die die TypeScript-Typprüfung in einer JavaScript-Datei aktiviert. Diese Direktive bietet eine weniger strenge Form der Typprüfung und kann zunächst dazu dienen, Probleme in JavaScript-Dateien zu erkennen. Wenn eine Datei `@ts-check` enthält, versucht TypeScript, Definitionen anhand von Kommentaren im JSDoc-Stil abzuleiten. Sie sollten JSDoc-Annotationen jedoch nur in einer sehr frühen Phase der Migration verwenden.

Erwägen Sie, den Standardwert von `noEmitOnError` in Ihrer tsconfig.json auf false zu belassen. So können Sie JavaScript-Quellcode auch dann ausgeben, wenn Fehler gemeldet werden.

Das Typsystem erkunden

Der TypeScript-Sprachdienst

Der TypeScript Language Service, auch als tsserver bekannt, bietet verschiedene Funktionen wie Fehlerberichte, Diagnosen, Kompilierung beim Speichern, Umbenennen, Springen zur Definition, Vervollständigungslisten, Signaturhilfe und mehr. Er

wird hauptsächlich von integrierten Entwicklungsumgebungen (IDEs) verwendet, um IntelliSense-Unterstützung bereitzustellen. Er lässt sich nahtlos in Visual Studio Code integrieren und wird von Werkzeugen wie Conquer of Completion (Coc) genutzt.

Entwickler können eine spezielle API nutzen und eigene Plugins für den Sprachdienst erstellen, um die Bearbeitung von TypeScript zu verbessern. Dies kann besonders nützlich sein, um spezielle Linting-Funktionen zu implementieren oder die automatische Vervollständigung für eine benutzerdefinierte Template-Sprache zu ermöglichen.

Ein Beispiel für ein benutzerdefiniertes Plugin aus der Praxis ist „typescript-styled-plugin“, das Syntaxfehlerberichte und IntelliSense-Unterstützung für CSS-Eigenschaften in Styled Components bereitstellt.

Weitere Informationen und Schnellstartanleitungen finden Sie im offiziellen TypeScript-Wiki auf GitHub: https://github.com/microsoft/TypeScript/wiki/

Strukturelle Typisierung

TypeScript basiert auf einem strukturellen Typsystem. Das bedeutet, dass die Kompatibilität und Gleichwertigkeit von Typen durch ihre tatsächliche Struktur oder Definition bestimmt werden und nicht durch ihren Namen oder den Ort ihrer Deklaration, wie dies bei nominalen Typsystemen etwa in C# oder C der Fall ist.

Das strukturelle Typsystem von TypeScript wurde nach dem Vorbild des dynamischen Duck-Typing-Systems von JavaScript

zur Laufzeit entwickelt.

Das folgende Beispiel ist gültiger TypeScript-Code. Wie Sie sehen können, besitzen „X“ und „Y“ denselben Member „a“, obwohl sie unterschiedliche Deklarationsnamen haben. Die Typen werden durch ihre Strukturen bestimmt. Da die Strukturen in diesem Fall identisch sind, sind sie kompatibel und gültig.

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // Valid
```

Grundlegende Vergleichsregeln von TypeScript

Der Vergleichsprozess von TypeScript erfolgt rekursiv und wird auf Typen angewendet, die auf beliebiger Ebene verschachtelt sind.

Ein Typ „X“ ist mit „Y“ kompatibel, wenn „Y“ mindestens dieselben Elemente wie „X“ besitzt.

```
type X = {
    a: string;
};
const y = { a: 'A', b: 'B' }; // Valid, as it has at least the
same members as X
const r: X = y;
```

Funktionsparameter werden anhand ihrer Typen und nicht anhand ihrer Namen verglichen:

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // Valid
x = y; // Valid
```

Die Rückgabetypen von Funktionen müssen identisch sein:

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Invalid
x = y; // Invalid
```

Der Rückgabetyp einer Quellfunktion muss ein Untertyp des Rückgabetyps einer Zielfunktion sein:

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Valid
y = x; // Invalid member b is missing
```

Das Verwerfen von Funktionsparametern ist zulässig, da dies in JavaScript gängige Praxis ist, beispielsweise bei der Verwendung von „Array.prototype.map()“:

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

Daher sind die folgenden Typdeklarationen vollständig gültig:

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Missing b parameter
y = x; // Valid
```

Alle zusätzlichen optionalen Parameter des Quelltyps sind gültig:

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; //Valid
```

Alle optionalen Parameter des Zieltyps ohne entsprechende Parameter im Quelltyp sind gültig und stellen keinen Fehler dar:

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; // Valid
```

Der Restparameter wird als unendliche Reihe optionaler Parameter behandelt:

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //valid
```

Funktionen mit Überladungen sind gültig, wenn die Überladungssignatur mit ihrer Implementierungssignatur kompatibel ist:

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Valid
```

```
x('a', 1); // Valid

function y(a: string): void; // Invalid, not compatible with
implementation signature
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}
y('a');
y('a', 1);
```

Der Vergleich von Funktionsparametern ist erfolgreich, wenn die Quell- und Zielparameter Ober- oder Untertypen zugewiesen werden können (Bivarianz).

```
// Supertype
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// Subtype
class Y extends X {}
// Subtype
class Z extends X {}

type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// Bivariance does accept supertypes
console.log(getA(new X('x'))); // Valid
console.log(getA(new Y('Y'))); // Valid
console.log(getA(new Z('z'))); // Valid
```

Enums können mit Zahlen verglichen und diesen zugewiesen werden und umgekehrt. Der Vergleich von Enum-Werten verschiedener Enum-Typen ist jedoch ungültig.

```
enum X {
    A,
    B,
}
enum Y {
    A,
    B,
    C,
}
const xa: number = X.A; // Valid
const ya: Y = 0; // Valid
X.A === Y.A; // Invalid
```

Bei Instanzen einer Klasse wird eine Kompatibilitätsprüfung ihrer privaten und geschützten Elemente durchgeführt:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
}

let x: X = new Y('y'); // Invalid
```

Die Vergleichsprüfung berücksichtigt unterschiedliche Vererbungshierarchien nicht, zum Beispiel:

```
class X {
    public a: string;
    constructor(value: string) {
```

```
        this.a = value;
    }
}
class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Valid
x === z; // Valid even if z is from a different inheritance hierarchy
```

Generische Typen werden anhand ihrer Strukturen verglichen, die sich nach Anwendung des generischen Parameters ergeben. Nur das Endergebnis wird als nicht generischer Typ verglichen.

```
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Invalid as the type argument is used in the final structure

interface X<T> {}
const x: X<number> = 1;
```

```
const y: X<string> = 'a';
x === y; // Valid as the type argument is not used in the final
structure
```

Wenn bei generischen Typen kein Typargument angegeben ist, werden alle nicht angegebenen Argumente als Typen mit „any“ behandelt:

```
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // Valid
```

Beachten Sie:

```
let a: number = 1;
let b: number = 2;
a = b; // Valid, everything is assignable to itself

let c: any;
c = 1; // Valid, all types are assignable to any

let d: unknown;
d = 1; // Valid, all types are assignable to unknown

let e: unknown;
let e1: unknown = e; // Valid, unknown is only assignable to
itself and any
let e2: any = e; // Valid
let e3: number = e; // Invalid

let f: never;
f = 1; // Invalid, nothing is assignable to never

let g: void;
let g1: any;
```

```
g = 1; // Invalid, void is not assignable to or from anything
except any
g = g1; // Valid
```

Beachten Sie, dass „null“ und „undefined“ bei aktiviertem „strictNullChecks“ ähnlich wie „void“ behandelt werden; andernfalls verhalten sie sich ähnlich wie „never“.

Typen als Mengen

In TypeScript ist ein Typ eine Menge möglicher Werte. Diese Menge wird auch als Wertebereich des Typs bezeichnet. Jeder Wert eines Typs kann als Element einer Menge betrachtet werden. Ein Typ legt die Bedingungen fest, die jedes Element der Menge erfüllen muss, um als Mitglied dieser Menge zu gelten. Die Hauptaufgabe von TypeScript besteht darin, zu prüfen und zu verifizieren, ob eine Menge eine Teilmenge einer anderen ist.

TypeScript unterstützt verschiedene Arten von Mengen:

Mengenbegriff	TypeScript	Hinweise
Leere Menge	never	„never“ enthält nichts außer sich selbst
Einelementige Menge	undefined / null / literal type	
Endliche Menge	boolean / union	
Unendliche Menge	string / number / object	
Universelle Menge	any / unknown	Jedes Element gehört zu „any“, und jede Menge ist eine Teilmenge davon / „unknown“ ist ein typsicheres Gegenstück zu „any“

Hier sind einige Beispiele:

TypeScript	Mengenbegriff	Beispiel
never	∅ (leere Menge)	const x: never = ‘x’; // Error: Type ‘string’ is not assignable to type ‘never’
Literaltyp	Einelementige Menge	type X = ‘X’; type Y = 7;
T zuweisbarer Wert	Wert ∈ T (Element von)	type XY = ‘X’ | ‘Y’; const x: XY = ‘X’;
T1 ist T2 zuweisbar	T1 ⊆ T2 (Teilmenge von)	type XY = ‘X’ | ‘Y’; const x: XY = ‘X’; const j: XY = ‘J’; // Type ‘“J”’ is not assignable to type ‘XY’.
T1 extends T2	T1 ⊆ T2 (Teilmenge von)	type X = ‘X’ extends string ? true : false;
T1 | T2	T1 ∪ T2 (Vereinigung)	type XY = ‘X’ | ‘Y’; type JK = 1 | 2;
T1 & T2	T1 ∩ T2 (Schnittmenge)	type X = { a: string } type Y = { b: string } type XY = X & Y const x: XY = { a: ‘a’, b: ‘b’ }
unknown	Universelle Menge	const x: unknown = 1

Eine Union (T1 | T2) erzeugt eine größere Menge (beide):

```
type X = {
    a: string;
};
```

```
type Y = {
    b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Valid
```

Eine Schnittmenge (T1 & T2) erzeugt eine kleinere Menge (nur gemeinsame Werte):

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Invalid
const j: XY = { a: 'a', b: 'b' }; // Valid
```

Das Schlüsselwort `extends` kann in diesem Zusammenhang als „Teilmenge von“ verstanden werden. Es legt eine Einschränkung für einen Typ fest. Wird `extends` mit einem generischen Typ verwendet, schränkt es den generischen Typparameter auf einen spezifischeren Typ ein.

Beachten Sie, dass `extends` hier nichts mit Klassenvererbung im Sinne der objektorientierten Programmierung zu tun hat.

TypeScript arbeitet mit strukturellen Typen und besitzt keine strikte nominale Hierarchie. Wie im folgenden Beispiel können sich zwei Typen überschneiden, ohne dass einer ein Untertyp des anderen ist, da TypeScript die Struktur beziehungsweise Form von Objekten berücksichtigt.

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Valid
```

Einen Typ zuweisen: Typdeklarationen und Typassertionen

In TypeScript kann ein Typ auf unterschiedliche Weise zugewiesen werden:

Typdeklaration

Im folgenden Beispiel verwenden wir x: X („: Type“), um einen Typ für die Variable x zu deklarieren.

```
type X = {
    a: string;
};

// Type declaration
const x: X = {
    a: 'a',
};
```

Wenn die Variable nicht dem angegebenen Format entspricht, meldet TypeScript einen Fehler. Zum Beispiel:

```
type X = {
    a: string;
};

const x: X = {
    a: 'a',
     b: 'b', // Error: Object literal may only specify known
properties
};
```

Typassertion

Mit dem Schlüsselwort `as` kann eine Typassertion hinzugefügt werden. Dadurch wird dem Compiler mitgeteilt, dass der Entwickler über zusätzliche Informationen zu einem Typ verfügt, und eventuell auftretende Fehler werden unterdrückt.

Zum Beispiel:

```
type X = {
    a: string;
};
const x = {
    a: 'a',
```

```
    b: 'b',
} as X;
```

Im obigen Beispiel wird mithilfe des Schlüsselworts as festgelegt, dass das Objekt x den Typ X besitzt. Dadurch wird dem TypeScript-Compiler mitgeteilt, dass das Objekt dem angegebenen Typ entspricht, obwohl es die zusätzliche Eigenschaft b besitzt, die in der Typdefinition nicht enthalten ist.

Typassertionen sind in Situationen nützlich, in denen ein spezifischerer Typ angegeben werden muss, insbesondere bei der Arbeit mit dem DOM. Zum Beispiel:

```
const myInput = document.getElementById('my_input') as
HTMLInputElement;
```

Hier wird die Typassertion as HTMLInputElement verwendet, um TypeScript mitzuteilen, dass das Ergebnis von getElementById als HTMLInputElement behandelt werden soll. Typassertionen können auch zum Neuzuordnen von Schlüsseln verwendet werden, wie das folgende Beispiel mit Template-Literalen zeigt:

```
type J<Type> = {
    [Property in keyof Type as `prefix_${string &
        Property}`]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
};
type Y = J<X>;
```

In diesem Beispiel verwendet der Typ `J<Type>` einen gemappten Typ mit einem Template-Literal, um die Schlüssel von Type neu

zuzuordnen. Er erstellt neue Eigenschaften, bei denen jedem Schlüssel „prefix_“ vorangestellt wird. Die zugehörigen Werte sind Funktionen, welche die ursprünglichen Eigenschaftswerte zurückgeben.

Beachten Sie, dass TypeScript bei Verwendung einer Typassertion keine Prüfung auf überschüssige Eigenschaften durchführt. Daher ist es im Allgemeinen vorzuziehen, eine Typdeklaration zu verwenden, wenn die Struktur des Objekts im Voraus bekannt ist.

Umgebungsdeklarationen

Umgebungsdeklarationen sind Dateien, die Typen für JavaScript-Code beschreiben; ihre Dateinamen haben das Format `.d.ts.`. Sie werden üblicherweise importiert und verwendet, um bestehende JavaScript-Bibliotheken zu annotieren oder vorhandenen JS-Dateien in Ihrem Projekt Typen hinzuzufügen.

Typen für viele gängige Bibliotheken finden Sie unter: https://github.com/DefinitelyTyped/DefinitelyTyped/

und können wie folgt installiert werden:

```
npm install --save-dev @types/library-name
```

Ihre selbst definierten Umgebungsdeklarationen können Sie über die „Triple-Slash“-Referenz importieren:

```
/// <reference path="./library-types.d.ts" />
```

Mit `// @ts-check` können Sie Umgebungsdeklarationen sogar innerhalb von JavaScript-Dateien verwenden.

Das Schlüsselwort `declare` ermöglicht Typdefinitionen für vorhandenen JavaScript-Code, ohne diesen zu importieren, und dient als Platzhalter für Typen aus einer anderen Datei oder aus dem globalen Gültigkeitsbereich.

Eigenschaftsprüfung und Prüfung auf überschüssige Eigenschaften

TypeScript basiert auf einem strukturellen Typsystem. Die Prüfung auf überschüssige Eigenschaften ist jedoch eine Funktion von TypeScript, mit der geprüft werden kann, ob ein Objekt genau die im Typ angegebenen Eigenschaften besitzt.

Die Prüfung auf überschüssige Eigenschaften wird beispielsweise durchgeführt, wenn Objektliterale Variablen zugewiesen oder als Argumente an Funktionen übergeben werden.

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
const x: X = y; // Valid because structural typing
const w: X = { a: 'a', b: 'b' }; // Invalid because excess
property checking
```

Schwache Typen

Ein Typ gilt als schwach, wenn er ausschließlich aus optionalen Eigenschaften besteht:

```
type X = {
    a?: string;
    b?: string;
};
```

TypeScript betrachtet eine Zuweisung an einen schwachen Typ als Fehler, wenn keine Überschneidung besteht. Das folgende Beispiel löst etwa einen Fehler aus:

```
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // Invalid
```

Obwohl dies nicht empfohlen wird, kann diese Prüfung bei Bedarf mithilfe einer Typassertion umgangen werden:

```
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Valid
```

Oder indem dem schwachen Typ eine Indexsignatur mit `unknown` hinzugefügt wird:

```
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Valid
```

Strikte Prüfung von Objektliteralen (Freshness)

Die strikte Prüfung von Objektliteralen, manchmal als „Freshness“ bezeichnet, ist eine TypeScript-Funktion, mit der überschüssige oder falsch geschriebene Eigenschaften erkannt werden können, die bei normalen strukturellen Typprüfungen andernfalls unbemerkt blieben.

Beim Erstellen eines Objektliterals betrachtet der TypeScript-Compiler dieses als „fresh“. Wird das Objektliteral einer Variablen zugewiesen oder als Parameter übergeben, gibt TypeScript einen Fehler aus, wenn das Objektliteral Eigenschaften angibt, die im Zieltyp nicht vorhanden sind.

„Freshness“ geht jedoch verloren, wenn ein Objektliteral erweitert oder eine Typassertion verwendet wird.

Die folgenden Beispiele veranschaulichen dies:

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Freshness check: Invalid assignment
var y: Y;
y = { a: 'a', bx: 'bx' }; // Freshness check: Invalid
assignment

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Widening: No errors, structurally type compatible

fn({ a: 'a', bx: 'b' }); // Freshness check: Invalid argument

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Widening: No Freshness check
```

Typinferenz

TypeScript kann Typen ableiten, wenn in folgenden Situationen keine Annotation angegeben wird:

- Initialisierung einer Variablen.
- Initialisierung eines Members.
- Festlegen von Standardwerten für Parameter.
- Rückgabetyp einer Funktion.

Zum Beispiel:

```
let x = 'x'; // The type inferred is string
```

Der TypeScript-Compiler analysiert den Wert oder Ausdruck und bestimmt dessen Typ anhand der verfügbaren Informationen.

Fortgeschrittenere Typinferenz

Wenn bei der Typinferenz mehrere Ausdrücke verwendet werden, sucht TypeScript nach den „besten gemeinsamen Typen“. Zum Beispiel:

```
let x = [1, 'x', 1, null]; // The type inferred is: (string | number | null)[]
```

Wenn der Compiler keine besten gemeinsamen Typen finden kann, gibt er einen Union-Typ zurück. Zum Beispiel:

```
let x = [new RegExp('x'), new Date()]; // Type inferred is: (RegExp | Date)[]
```

TypeScript nutzt „kontextuelle Typisierung“, um Typen anhand der Position einer Variablen abzuleiten. Im folgenden Beispiel

weiß der Compiler, dass `e` vom Typ `MouseEvent` ist. Grundlage dafür ist der in der Datei lib.d.ts definierte Ereignistyp `click`; diese Datei enthält Umgebungsdeklarationen für verschiedene gängige JavaScript-Konstrukte und das DOM:

```
window.addEventListener('click', function (e) {}); // The inferred type of e is MouseEvent
```

Typerweiterung

Bei der Typerweiterung weist TypeScript einer ohne Typannotation initialisierten Variablen einen Typ zu. Sie ermöglicht den Übergang von engeren zu weiteren Typen, jedoch nicht umgekehrt. Im folgenden Beispiel:

```
let x = 'x'; // TypeScript infers as string, a wide type
let y: 'y' | 'x' = 'y'; // y types is a union of literal types
y = x; // Invalid Type 'string' is not assignable to type '"x" | "y"'.
```

TypeScript weist den Typ `string` der Variablen `x` anhand des einzelnen bei der Initialisierung angegebenen Werts (`x`) zu. Dies ist ein Beispiel für eine Typerweiterung.

TypeScript bietet Möglichkeiten, den Erweiterungsprozess zu steuern, beispielsweise mithilfe von „const“.

Const

Die Verwendung des Schlüsselworts `const` bei der Deklaration einer Variablen führt in TypeScript zu einer engeren Typinferenz.

Zum Beispiel:

```
const x = 'x'; // TypeScript infers the type of x as 'x', a
narrower type
let y: 'y' | 'x' = 'y';
y = x; // Valid: The type of x is inferred as 'x'
```

Durch die Deklaration der Variablen x mit `const` wird ihr Typ auf den konkreten Literalwert ‘x’ eingegrenzt. Da der Typ von x eingegrenzt ist, kann er der Variablen y fehlerfrei zugewiesen werden. Der Typ kann abgeleitet werden, weil `const`-Variablen nicht neu zugewiesen werden können. Ihr Typ kann daher auf einen bestimmten Literaltyp eingegrenzt werden, in diesem Fall auf den Literaltyp ‘x’.

Const-Modifikator für Typparameter

Ab TypeScript-Version 5.0 kann das Attribut `const` für einen generischen Typparameter angegeben werden. Dadurch lässt sich der genauestmögliche Typ ableiten. Betrachten wir ein Beispiel ohne `const`:

```
function identity<T>(value: T) {
    // No const here
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Type inferred
is: { a: string; b: string; }
```

Wie Sie sehen, wird für die Eigenschaften `a` und `b` der Typ `string` abgeleitet.

Betrachten wir nun den Unterschied bei der Variante mit `const`:

```
function identity<const T>(value: T) {
    // Using const modifier on type parameters
    return value;
```

```
}
const values = identity({ a: 'a', b: 'b' }); // Type inferred
is: { a: "a"; b: "b"; }
```

Nun ist zu erkennen, dass die Eigenschaften `a` und `b` als Zeichenkettenliterale statt lediglich als `string`-Typen abgeleitet werden.

Const-Assertion

Mit dieser Funktion können Sie eine Variable anhand ihres Initialisierungswerts mit einem genaueren Literaltyp deklarieren und dem Compiler damit mitteilen, dass der Wert als unveränderliches Literal behandelt werden soll. Hier sind einige Beispiele:

Für eine einzelne Eigenschaft:

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

Für ein gesamtes Objekt:

```
const v = {
    x: 1,
    y: 2,
} as const;
```

Dies kann bei der Definition des Typs für ein Tupel besonders nützlich sein:

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // Tuple of readonly [1, 2, 3]
```

Explizite Typannotation

Wir können einen Typ explizit angeben. Im folgenden Beispiel ist die Eigenschaft `x` vom Typ `number`:

```
const v = {
    x: 1, // Inferred type: number (widening)
};
v.x = 3; // Valid
```

Mithilfe einer Union von Literaltypen können wir die Typannotation spezifischer gestalten:

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // x is now a union of literal types: 1 | 2 | 3
};
v.x = 3; // Valid
v.x = 100; // Invalid
```

Type Narrowing

Type Narrowing bezeichnet in TypeScript den Vorgang, bei dem ein allgemeiner Typ auf einen spezifischeren Typ eingegrenzt wird. Dies geschieht, wenn TypeScript den Code analysiert und feststellt, dass bestimmte Bedingungen oder Operationen die Typinformationen präzisieren können.

Typen können unter anderem auf folgende Arten eingegrenzt werden:

Bedingungen

Mithilfe bedingter Anweisungen wie `if` oder `switch` kann TypeScript den Typ anhand des Ergebnisses der Bedingung eingrenzen. Zum Beispiel:

```
let x: number | undefined = 10;

if (x !== undefined) {
    x += 100; // The type is number, which had been narrowed by
the condition
}
```

Auslösen eines Fehlers oder Zurückkehren

Das Auslösen eines Fehlers oder das vorzeitige Zurückkehren aus einem Zweig kann TypeScript dabei helfen, einen Typ einzugrenzen. Zum Beispiel:

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

Weitere Möglichkeiten zur Eingrenzung von Typen in TypeScript sind:

- Operator `instanceof`: Prüft, ob ein Objekt eine Instanz einer bestimmten Klasse ist.
- Operator `in`: Prüft, ob eine Eigenschaft in einem Objekt vorhanden ist.
- Operator `typeof`: Prüft den Typ eines Werts zur Laufzeit.
- Integrierte Funktionen wie `Array.isArray()`: Prüfen, ob ein Wert ein Array ist.

Diskriminierte Union

Eine „diskriminierte Union“ ist ein Muster in TypeScript, bei dem Objekten ein explizites „Tag“ hinzugefügt wird, um

verschiedene Typen innerhalb einer Union zu unterscheiden. Dieses Muster wird auch als „Tagged Union“ bezeichnet. Im folgenden Beispiel wird das „Tag“ durch die Eigenschaft „type“ dargestellt:

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // type is A
        case 'type_b':
            return input.value + 'extra'; // type is B
    }
};
```

Benutzerdefinierte Type Guards

Wenn TypeScript einen Typ nicht bestimmen kann, lässt sich eine Hilfsfunktion schreiben, die als „benutzerdefinierter Type Guard“ bezeichnet wird. Im folgenden Beispiel verwenden wir ein Typprädikat, um den Typ nach einer bestimmten Filterung einzugrenzen:

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // The type is (string
| null)[], TypeScript was not able to infer the type properly

const isValid = (item: string | null): item is string => item
!== null; // Custom type guard

const  r2  =  data.filter(isValid);  //  The  type  is  fine  now
string[],  by  using  the  predicate  type  guard  we  were  able  to
narrow the type
```

Switch-true Narrowing

TypeScript 5.3 führt Switch-true Narrowing ein. Damit können unübersichtliche if/else-Ketten durch switch (true) mit booleschen Bedingungen ersetzt werden. Dies verbessert die Lesbarkeit und grenzt die Typen weiterhin ein. Es ähnelt dem Pattern Matching, ist jedoch einfacher.

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

Primitive Typen

TypeScript unterstützt 7 primitive Typen. Ein primitiver Datentyp bezeichnet einen Typ, der kein Objekt ist und dem keine Methoden zugeordnet sind. In TypeScript sind alle primitiven Typen unveränderlich, das heißt, ihre Werte können nach der Zuweisung nicht mehr geändert werden.

string

Der primitive Typ `string` speichert Textdaten, und der Wert steht immer in doppelten oder einfachen Anführungszeichen.

```
const x: string = 'x';
const y: string = 'y';
```

Strings können sich über mehrere Zeilen erstrecken, wenn sie vom Backtick-Zeichen (`) umschlossen sind:

```
let sentence: string = `xxx,
   yyy`;
```

boolean

Der Datentyp `boolean` speichert in TypeScript einen binären Wert, entweder `true` oder `false`.

```
const isReady: boolean = true;
```

number

Ein `number`-Datentyp wird in TypeScript durch einen 64-Bit-Gleitkommawert dargestellt. Ein `number`-Typ kann ganze Zahlen und Brüche darstellen. TypeScript unterstützt beispielsweise auch hexadezimale, binäre und oktale Zahlen:

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Hexadecimal starts with
0x
const binary: number = 0b1010; // Binary starts with 0b
const octal: number = 0o633; // Octal starts with 0o
```

bigint

Ein `bigint` stellt Ganzzahlwerte dar, die größer als die von `number` unterstützte größte sichere Ganzzahl sein können, nämlich 2^53 - 1.

Ein `bigint` kann durch Aufrufen der integrierten Funktion `BigInt()` oder durch Anhängen von `n` an ein beliebiges ganzzahliges numerisches Literal erstellt werden:

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

Hinweise:

- `bigint`-Werte können nicht mit `number` gemischt und nicht mit dem integrierten `Math` verwendet werden; sie müssen in denselben Typ konvertiert werden.
- `bigint`-Werte sind nur verfügbar, wenn die Zielkonfiguration ES2020 oder höher ist.

Symbol

Symbole sind eindeutige Bezeichner, die als Eigenschaftsschlüssel in Objekten verwendet werden können, um Namenskonflikte zu vermeiden.

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null und undefined

Die Typen `null` und `undefined` stellen beide keinen Wert beziehungsweise das Fehlen eines Werts dar.

Der Typ `undefined` bedeutet, dass der Wert nicht zugewiesen oder initialisiert wurde oder dass ein Wert unbeabsichtigt fehlt.

Der Typ `null` bedeutet, dass bekannt ist, dass das Feld keinen Wert besitzt und der Wert daher nicht verfügbar ist; er weist auf das beabsichtigte Fehlen eines Werts hin.

Array

Ein `array` ist ein Datentyp, der mehrere Werte desselben oder unterschiedlicher Typen speichern kann. Er kann mit der folgenden Syntax definiert werden:

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // Union
```

TypeScript unterstützt schreibgeschützte Arrays mit der folgenden Syntax:

```
const x: readonly string[] = ['a', 'b']; // Readonly modifier
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Invalid
```

TypeScript unterstützt Tupel und schreibgeschützte Tupel:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

Der Datentyp `any` stellt wörtlich einen beliebigen Wert dar und ist der Standard, wenn TypeScript den Typ nicht ableiten kann oder er nicht angegeben ist.

Bei Verwendung von `any` überspringt der TypeScript-Compiler die Typprüfung, sodass bei der Verwendung von `any` keine Typsicherheit besteht. Verwenden Sie `any` im Allgemeinen nicht, um den Compiler beim Auftreten eines Fehlers stummzuschalten. Konzentrieren Sie sich stattdessen darauf, den Fehler zu beheben, da `any` dazu führen kann, dass Verträge verletzt werden und die Vorteile der TypeScript-Autovervollständigung verloren gehen.

Der Typ `any` kann bei einer schrittweisen Migration von JavaScript zu TypeScript nützlich sein, da er den Compiler stummschalten kann.

Verwenden Sie für neue Projekte die TypeScript-Konfiguration `noImplicitAny`, durch die TypeScript Fehler ausgibt, wenn `any` verwendet oder abgeleitet wird.

Der Typ `any` ist häufig eine Fehlerquelle, die echte Probleme mit Ihren Typen verschleiern kann. Vermeiden Sie seine Verwendung so weit wie möglich.

Typannotationen

Bei Variablen, die mit `var`, `let` und `const` deklariert werden, kann optional ein Typ hinzugefügt werden:

```
const x: number = 1;
```

TypeScript kann Typen, insbesondere einfache, gut ableiten, sodass diese Deklarationen in den meisten Fällen nicht erforderlich sind.

Bei Funktionen können Parametern Typannotationen hinzugefügt werden:

```
function sum(a: number, b: number) {
    return a + b;
}
```

Das folgende Beispiel verwendet eine anonyme Funktion (auch Lambda-Funktion genannt):

```
const sum = (a: number, b: number) => a + b;
```

Diese Annotationen können entfallen, wenn für einen Parameter ein Standardwert vorhanden ist:

```
const sum = (a = 10, b: number) => a + b;
```

Funktionen können mit Rückgabetypannotationen versehen werden:

```
const sum = (a = 10, b: number): number => a + b;
```

Dies ist besonders bei komplexeren Funktionen nützlich, da das Festlegen des Rückgabetyps vor der Implementierung dabei helfen kann, die Funktion zu durchdenken.

Erwägen Sie im Allgemeinen, Typsignaturen zu annotieren, jedoch keine lokalen Variablen im Funktionsrumpf, und fügen Sie Objektliteralen immer Typen hinzu.

Optionale Eigenschaften

Ein Objekt kann optionale Eigenschaften angeben, indem dem Eigenschaftsnamen ein Fragezeichen `?` angehängt wird:

```
type X = {
    a: number;
    b?: number; // Optional
};
```

Wenn eine Eigenschaft optional ist, kann ein Standardwert angegeben werden:

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

Schreibgeschützte Eigenschaften

Mit dem Modifizierer `readonly` kann das Schreiben in eine Eigenschaft verhindert werden. Er stellt sicher, dass der Eigenschaft kein neuer Wert zugewiesen werden kann, bietet jedoch keine Garantie für vollständige Unveränderlichkeit:

```
interface Y {
    readonly a: number;
}

type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;
```

```
type K = {
    readonly [index: number]: string;
};
```

Indexsignaturen

In TypeScript können wir `string`, `number` und `symbol` als Indexsignaturen verwenden:

```
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // Same result as k[1]
```

Beachten Sie, dass JavaScript einen Index vom Typ `number` automatisch in einen Index vom Typ `string` konvertiert, sodass `k[1]` und `k["1"]` denselben Wert zurückgeben.

Erweitern von Typen

Ein `interface` kann erweitert werden (Member eines anderen Typs werden kopiert):

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

Es ist auch möglich, mehrere Typen zu erweitern:

```
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

Das Schlüsselwort `extends` funktioniert nur bei Interfaces und Klassen; verwenden Sie für Typen eine Schnittmenge:

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

Ein Typ kann mit einem Interface erweitert werden, umgekehrt ist dies jedoch nicht möglich:

```
type A = {
    a: string;
};
interface B extends A {
    b: string;
}
```

Literaltypen

Ein Literaltyp ist eine einelementige Menge innerhalb eines umfassenden Typs; er definiert einen sehr genauen Wert, der ein JavaScript-Primitivwert ist.

Literaltypen in TypeScript sind Zahlen, Strings und boolesche Werte.

Beispiel für Literale:

```
const a = 'a'; // String literal type
const b = 1; // Numeric literal type
const c = true; // Boolean literal type
```

String-, numerische und boolesche Literaltypen werden in Unions, Type Guards und Typaliasen verwendet. Im folgenden Beispiel sehen Sie einen Union-Typalias. `O` besteht nur aus den angegebenen Werten; kein anderer String ist gültig:

```
type O = 'a' | 'b' | 'c';
```

Literaltypinferenz

Die Literaltypinferenz ist eine Funktion von TypeScript, mit der der Typ einer Variablen oder eines Parameters anhand seines Werts abgeleitet werden kann.

Im folgenden Beispiel sehen wir, dass TypeScript `x` als Literaltyp betrachtet, da der Wert später nicht geändert werden kann. `y` wird dagegen als String abgeleitet, da der Wert später jederzeit geändert werden kann.

```
const x = 'x'; // Literal type of 'x', because this value cannot be changed
let y = 'y'; // Type string, as we can change this value
```

Im folgenden Beispiel sehen wir, dass `o.x` als `string` (und nicht als Literal von `a`) abgeleitet wurde, da TypeScript davon ausgeht, dass der Wert später jederzeit geändert werden kann.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // This is a wider string
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x)); // Argument of type 'string' is not
assignable to parameter of type 'X'
```

Wie Sie sehen, gibt der Code beim Übergeben von `o.x` an `fn` einen Fehler aus, da X ein engerer Typ ist.

Dieses Problem kann durch eine Typassertion mit `const` oder dem Typ `X` gelöst werden:

```
let o = {
    x: 'a' as const,
};
```

oder:

```
let o = {
    x: 'a' as X,
};
```

strictNullChecks

`strictNullChecks` ist eine TypeScript-Compileroption, die eine strikte Nullprüfung erzwingt. Wenn diese Option aktiviert ist, können `null` oder `undefined` Variablen und Parametern nur dann zugewiesen werden, wenn diese ausdrücklich mit dem Union-Typ `null | undefined` deklariert wurden. Wenn eine Variable oder ein Parameter nicht ausdrücklich als nullable

deklariert ist, erzeugt TypeScript einen Fehler, um potenzielle Laufzeitfehler zu verhindern.

Enums

In TypeScript ist ein `enum` eine Menge benannter konstanter Werte.

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

Enums können auf verschiedene Arten definiert werden:

Numerische Enums

In TypeScript ist ein numerisches Enum ein Enum, bei dem jeder Konstanten ein numerischer Wert zugewiesen wird, der standardmäßig bei 0 beginnt.

```
enum Size {
    Small, // value starts from 0
    Medium,
    Large,
}
```

Durch eine explizite Zuweisung können benutzerdefinierte Werte angegeben werden:

```
enum Size {
    Small = 10,
    Medium,
    Large,
```

```
}
console.log(Size.Medium); // 11
```

String-Enums

In TypeScript ist ein String-Enum ein Enum, bei dem jeder Konstanten ein Stringwert zugewiesen wird.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

Hinweis: TypeScript erlaubt die Verwendung heterogener Enums, in denen String- und numerische Member nebeneinander vorhanden sein können.

Konstante Enums

Ein konstantes Enum ist in TypeScript ein besonderer Enum-Typ, bei dem alle Werte zur Kompilierzeit bekannt sind und überall dort inline eingefügt werden, wo das Enum verwendet wird, was zu effizienterem Code führt.

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

Wird kompiliert zu:

```
console.log('EN' /* Language.English */);
```

Hinweise: Const-Enums besitzen fest codierte Werte, wodurch das Enum entfernt wird. Dies kann in eigenständigen

Bibliotheken effizienter sein, ist im Allgemeinen jedoch nicht wünschenswert. Außerdem können Const-Enums keine berechneten Member enthalten.

Reverse Mapping

In TypeScript bezeichnen Reverse Mappings in Enums die Möglichkeit, den Namen eines Enum-Members anhand seines Werts abzurufen. Standardmäßig besitzen Enum-Member Vorwärtszuordnungen vom Namen zum Wert. Reverse Mappings können jedoch erstellt werden, indem für jeden Member explizit Werte festgelegt werden. Reverse Mappings sind nützlich, wenn Sie einen Enum-Member anhand seines Werts nachschlagen oder über alle Enum-Member iterieren müssen. Beachten Sie, dass nur numerische Enum-Member Reverse Mappings erzeugen, während für String-Enum-Member überhaupt kein Reverse Mapping erzeugt wird.

Das folgende Enum:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
```

Wird kompiliert zu:

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
```

```
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

Daher funktioniert die Zuordnung von Werten zu Schlüsseln für numerische Enum-Member, nicht jedoch für String-Enum-Member:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]);  //  Element  implicitly  has  an
'any' type because index expression is not of type 'number'.
```

Ambient Enums

Ein Ambient Enum ist in TypeScript ein Enum-Typ, der in einer Deklarationsdatei (*.d.ts) ohne zugehörige Implementierung definiert wird. Damit können Sie eine Menge benannter Konstanten definieren, die typsicher in verschiedenen Dateien verwendet werden können, ohne die Implementierungsdetails in jede Datei importieren zu müssen.

Berechnete und konstante Member

In TypeScript ist ein berechneter Member ein Member eines Enums, dessen Wert zur Laufzeit berechnet wird. Ein konstanter Member ist dagegen ein Member, dessen Wert zur

Kompilierzeit festgelegt wird und zur Laufzeit nicht geändert werden kann. Berechnete Member sind in regulären Enums zulässig, während konstante Member sowohl in regulären als auch in Const-Enums zulässig sind.

```
// Constant members
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6 generation at compilation time

// Computed members
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // random number generated at run time
```

Enums werden durch Unions dargestellt, die aus den Typen ihrer Member bestehen. Die Werte der einzelnen Member können durch konstante oder nicht konstante Ausdrücke bestimmt werden, wobei Membern mit konstanten Werten Literaltypen zugewiesen werden. Betrachten Sie zur Veranschaulichung die Deklaration des Typs E und seiner Untertypen E.A, E.B und E.C. In diesem Fall stellt E die Union E.A | E.B | E.C dar.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // Numeric literal
    B = 'bar', // String literal
    C = identity(42), // Opaque computed
```

```
}

console.log(E.C); //42
```

Narrowing

TypeScript-Narrowing ist der Prozess, bei dem der Typ einer Variablen innerhalb eines bedingten Blocks präzisiert wird. Dies ist bei der Arbeit mit Union-Typen nützlich, bei denen eine Variable mehr als einen Typ haben kann.

TypeScript erkennt mehrere Möglichkeiten, einen Typ einzugrenzen:

typeof Type Guards

Der typeof-Type-Guard ist ein bestimmter Type Guard in TypeScript, der den Typ einer Variablen anhand ihres integrierten JavaScript-Typs prüft.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x is number
    }
    return -1;
};
```

Truthiness-Narrowing

Truthiness-Narrowing funktioniert in TypeScript, indem geprüft wird, ob eine Variable truthy oder falsy ist, um ihren Typ entsprechend einzugrenzen.

```
const toUpperCase = (name: string | null) => {
    if (name) {
```

```
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

Equality-Narrowing

Equality-Narrowing funktioniert in TypeScript, indem geprüft wird, ob eine Variable einem bestimmten Wert entspricht, um ihren Typ entsprechend einzugrenzen.

Es wird zusammen mit `switch`-Anweisungen und Gleichheitsoperatoren wie `===`, `!==`, `==` und `!=` verwendet, um Typen einzugrenzen.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

Narrowing mit dem in-Operator

Das Narrowing mit dem `in`-Operator ist in TypeScript eine Möglichkeit, den Typ einer Variablen danach einzugrenzen, ob in ihrem Typ eine Eigenschaft vorhanden ist.

```
type Dog = {
    name: string;
    breed: string;
};
```

```
type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

Narrowing mit instanceof

Das Narrowing mit dem `instanceof`-Operator ist in TypeScript eine Möglichkeit, den Typ einer Variablen anhand ihrer Konstruktorfunktion einzugrenzen, indem geprüft wird, ob ein Objekt eine Instanz einer bestimmten Klasse oder eines bestimmten Interfaces ist.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
```

```
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

Zuweisungen

TypeScript-Narrowing mithilfe von Zuweisungen ist eine Möglichkeit, den Typ einer Variablen anhand des ihr zugewiesenen Werts einzugrenzen. Wenn einer Variablen ein Wert zugewiesen wird, leitet TypeScript ihren Typ anhand des zugewiesenen Werts ab und grenzt den Typ der Variablen entsprechend dem abgeleiteten Typ ein.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

Kontrollflussanalyse

Die Kontrollflussanalyse ist in TypeScript eine Möglichkeit, den Codefluss statisch zu analysieren, um die Typen von Variablen abzuleiten. Dadurch kann der Compiler die Typen dieser Variablen anhand der Analyseergebnisse nach Bedarf eingrenzen.

Vor TypeScript 4.4 wurde die Kontrollflussanalyse nur auf Code innerhalb einer if-Anweisung angewendet. Seit TypeScript 4.4

kann sie auch auf bedingte Ausdrücke und Zugriffe auf Diskriminanteigenschaften angewendet werden, auf die indirekt über const-Variablen verwiesen wird.

Zum Beispiel:

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

Einige Beispiele, in denen kein Narrowing erfolgt:

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
        x.length; // Error, no narrowing because isString it is
not const
    }
};

const f6 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
```

```
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
        obj.foo; // Error, no narrowing because obj is assigned
in function body
    }
};
```

Hinweise: In bedingten Ausdrücken werden bis zu fünf Indirektionsebenen analysiert.

Typprädikate

Typprädikate sind in TypeScript Funktionen, die einen booleschen Wert zurückgeben und verwendet werden, um den Typ einer Variablen auf einen spezifischeren Typ einzugrenzen.

```
const isString = (value: unknown): value is string => typeof
value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
    }
};
```

TypeScript 5.5 leitet Typprädikate (wie `x is T`) in Funktionen wie `.filter` automatisch ab. Dadurch erkennt es, wann Werte wie undefined entfernt werden, was zu präziseren Typen und weniger Fehlern führt. Dies funktioniert bei eindeutigen Prüfungen (z. B. `x !== undefined`), nicht jedoch bei mehrdeutigen wie `!!x`.

```ts
const nums = [1, null, 2].filter(x => x !== null);
```

Diskriminierte Unions

Diskriminierte Unions sind in TypeScript Union-Typen, die eine gemeinsame Eigenschaft, die sogenannte Diskriminante, verwenden, um die Menge der möglichen Typen der Union einzugrenzen.

```ts
type Square = {
    kind: 'square'; // Discriminant
    size: number;
};

type Circle = {
    kind: 'circle'; // Discriminant
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

Der Typ never

Wenn eine Variable auf einen Typ eingegrenzt wird, der keine Werte enthalten kann, leitet der TypeScript-Compiler ab, dass die Variable vom Typ `never` sein muss. Dies liegt daran, dass der Typ never einen Wert darstellt, der niemals erzeugt werden kann.

```
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
          // val has type never here because it can never be
anything other than a string or a number
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};
```

Vollständigkeitsprüfung

Die Vollständigkeitsprüfung ist eine Funktion von TypeScript, die sicherstellt, dass alle möglichen Fälle einer diskriminierten Union in einer `switch`- oder `if`-Anweisung behandelt werden.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
            console.log('Moving down');
```

```
            break;
        default:
            const exhaustiveCheck: never = direction;
                console.log(exhaustiveCheck); // This line will
never be executed
    }
};
```

Der Typ `never` wird verwendet, um sicherzustellen, dass der Standardfall vollständig ist und TypeScript einen Fehler ausgibt, wenn dem Typ Direction ein neuer Wert hinzugefügt wird, ohne dass dieser in der switch-Anweisung behandelt wird.

Objekttypen

In TypeScript beschreiben Objekttypen die Struktur eines Objekts. Sie geben die Namen und Typen der Eigenschaften des Objekts sowie an, ob diese Eigenschaften erforderlich oder optional sind.

In TypeScript können Sie Objekttypen auf zwei grundlegende Arten definieren:

Ein Interface definiert die Struktur eines Objekts, indem es die Namen, Typen und Optionalität seiner Eigenschaften angibt.

```
interface User {
    name: string;
    age: number;
    email?: string;
}
```

Ein Typalias definiert ähnlich wie ein Interface die Struktur eines Objekts. Er kann jedoch auch einen neuen

benutzerdefinierten Typ erstellen, der auf einem bestehenden Typ oder einer Kombination bestehender Typen basiert. Dazu gehört die Definition von Union-Typen, Schnittmengentypen und anderen komplexen Typen.

```
type Point = {
    x: number;
    y: number;
};
```

Ein Typ kann auch anonym definiert werden:

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

Tupeltyp (anonym)

Ein Tupeltyp ist ein Typ, der ein Array mit einer festen Anzahl von Elementen und den zugehörigen Typen darstellt. Ein Tupeltyp erzwingt eine bestimmte Anzahl von Elementen und deren jeweilige Typen in einer festen Reihenfolge. Tupeltypen sind nützlich, wenn Sie eine Sammlung von Werten mit bestimmten Typen darstellen möchten, bei der die Position jedes Elements im Array eine bestimmte Bedeutung hat.

```
type Point = [number, number];
```

Benannter Tupeltyp (beschriftet)

Tupeltypen können optionale Beschriftungen oder Namen für jedes Element enthalten. Diese Beschriftungen dienen der Lesbarkeit und der Unterstützung durch Werkzeuge und wirken sich nicht auf die Operationen aus, die Sie mit ihnen ausführen können.

```
type T = string;
type Tuple1 = [T, T];
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // Named Tuple plus Anonymous Tuple
```

Tupel mit fester Länge

Ein Tupel mit fester Länge ist eine bestimmte Art von Tupel, die eine feste Anzahl von Elementen bestimmter Typen erzwingt und nach der Definition keine Änderungen an der Länge des Tupels zulässt.

Tupel mit fester Länge sind nützlich, wenn Sie eine Sammlung von Werten mit einer bestimmten Anzahl von Elementen und bestimmten Typen darstellen müssen und sicherstellen möchten, dass die Länge und die Typen des Tupels nicht versehentlich geändert werden können.

```
const x = [10, 'hello'] as const;
x.push(2); // Error
```

Union-Typ

Ein Union-Typ ist ein Typ, der einen Wert darstellt, der einem von mehreren Typen angehören kann. Union-Typen werden durch das Symbol | zwischen den einzelnen möglichen Typen gekennzeichnet.

```
let x: string | number;
x = 'hello'; // Valid
x = 123; // Valid
```

Schnittmengentypen

Ein Schnittmengentyp ist ein Typ, der einen Wert darstellt, der alle Eigenschaften von zwei oder mehr Typen besitzt. Schnittmengentypen werden durch das Symbol & zwischen den einzelnen Typen gekennzeichnet.

```
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // Intersection

const j: J = {
    a: 'a',
    b: 'b',
};
```

Typindizierung

Typindizierung bezeichnet die Möglichkeit, Typen zu definieren, die mit einem nicht im Voraus bekannten Schlüssel indiziert werden können. Dabei wird eine Indexsignatur verwendet, um den Typ für Eigenschaften anzugeben, die nicht explizit deklariert sind.

```
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // Returns a
```

Typ aus einem Wert

Ein Typ aus einem Wert bezeichnet in TypeScript die automatische Ableitung eines Typs aus einem Wert oder Ausdruck durch Typinferenz.

```
const x = 'x'; // TypeScript infers 'x' as a string literal with 'const' (immutable), but widens it to 'string' with 'let' (reassignable).
```

Typ aus dem Funktionsrückgabewert

Ein Typ aus dem Funktionsrückgabewert bezeichnet die Möglichkeit, den Rückgabetyp einer Funktion anhand ihrer Implementierung automatisch abzuleiten. Dadurch kann TypeScript den Typ des von der Funktion zurückgegebenen Werts ohne explizite Typannotationen bestimmen.

```
const add = (x: number, y: number) => x + y; // TypeScript can infer that the return type of the function is a number
```

Typ aus einem Modul

Ein Typ aus einem Modul bezeichnet die Möglichkeit, die exportierten Werte eines Moduls zu verwenden, um deren Typen automatisch abzuleiten. Wenn ein Modul einen Wert mit einem bestimmten Typ exportiert, kann TypeScript diese Information verwenden, um den Typ dieses Werts automatisch abzuleiten, wenn er in ein anderes Modul importiert wird.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r is number
```

Gemappte Typen

Gemappte Typen ermöglichen es in TypeScript, neue Typen auf Grundlage eines vorhandenen Typs zu erstellen, indem jede Eigenschaft mithilfe einer Mapping-Funktion transformiert wird. Durch das Mapping vorhandener Typen können Sie neue Typen erstellen, die dieselben Informationen in einem anderen Format darstellen. Um einen gemappten Typ zu erstellen, greifen Sie mit dem Operator `keyof` auf die Eigenschaften eines vorhandenen Typs zu und verändern sie anschließend, um einen neuen Typ zu erzeugen. Im folgenden Beispiel:

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

Wir definieren MyMappedType so, dass die Eigenschaften von T gemappt werden und ein neuer Typ entsteht, bei dem jede Eigenschaft ein Array ihres ursprünglichen Typs ist. Damit erstellen wir MyNewType, um dieselben Informationen wie MyType darzustellen, jedoch mit jeder Eigenschaft als Array.

Modifizierer gemappter Typen

Modifizierer gemappter Typen ermöglichen in TypeScript die Transformation von Eigenschaften innerhalb eines vorhandenen Typs:

- `readonly` oder `+readonly`: Dadurch wird eine Eigenschaft im gemappten Typ schreibgeschützt.
- `-readonly`: Dadurch wird eine Eigenschaft im gemappten Typ veränderbar.
- `?`: Dadurch wird eine Eigenschaft im gemappten Typ als optional gekennzeichnet.

Beispiele:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // All properties marked as read-only

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // All properties marked as mutable

type MyPartial<T> = { [P in keyof T]?: T[P] }; // All properties marked as optional
```

Bedingte Typen

Bedingte Typen sind eine Möglichkeit, einen von einer Bedingung abhängigen Typ zu erstellen, wobei der zu erstellende Typ anhand des Ergebnisses der Bedingung bestimmt wird. Sie werden mit dem Schlüsselwort `extends` und einem ternären Operator definiert, um bedingt zwischen zwei Typen zu wählen.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;
```

```
type IsMyArrayAnArray = IsArray<typeof myArray>; // Type true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Type
false
```

Distributive bedingte Typen

Distributive bedingte Typen sind eine Funktion, mit der ein Typ über eine Union von Typen verteilt werden kann, indem auf jeden Member der Union einzeln eine Transformation angewendet wird. Dies kann besonders bei der Arbeit mit gemappten Typen oder Typen höherer Ordnung nützlich sein.

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number |
boolean | null
```

Typinferenz mit infer in bedingten Typen

Das Schlüsselwort `infer` wird in bedingten Typen verwendet, um den Typ eines generischen Parameters aus einem von ihm abhängigen Typ abzuleiten (zu extrahieren). Dadurch können Sie flexiblere und wiederverwendbare Typdefinitionen schreiben.

```
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

Vordefinierte bedingte Typen

Vordefinierte bedingte Typen sind in TypeScript integrierte bedingte Typen, die von der Sprache bereitgestellt werden. Sie

sind dafür vorgesehen, gängige Typtransformationen anhand der Eigenschaften eines bestimmten Typs durchzuführen.

`Exclude<UnionType, ExcludedType>`: Dieser Typ entfernt alle Typen aus Type, die ExcludedType zugewiesen werden können.

`Extract<Type, Union>`: Dieser Typ extrahiert alle Typen aus Union, die Type zugewiesen werden können.

`NonNullable<Type>`: Dieser Typ entfernt null und undefined aus Type.

`ReturnType<Type>`: Dieser Typ extrahiert den Rückgabetyp einer Funktion Type.

`Parameters<Type>`: Dieser Typ extrahiert die Parametertypen einer Funktion Type.

`Required<Type>`: Dieser Typ macht alle Eigenschaften in Type erforderlich.

`Partial<Type>`: Dieser Typ macht alle Eigenschaften in Type optional.

`Readonly<Type>`: Dieser Typ macht alle Eigenschaften in Type schreibgeschützt.

Template-Union-Typen

Template-Union-Typen können beispielsweise verwendet werden, um Text innerhalb des Typsystems zusammenzuführen und zu bearbeiten:

```ts
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active" | "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

Typ any

Der Typ `any` ist ein spezieller Typ (universeller Supertyp), der zur Darstellung jedes beliebigen Werttyps (Primitivwerte, Objekte, Arrays, Funktionen, Fehler, Symbole) verwendet werden kann. Er wird häufig in Situationen verwendet, in denen der Typ eines Werts zur Kompilierzeit nicht bekannt ist oder wenn mit Werten aus externen APIs oder Bibliotheken gearbeitet wird, die keine TypeScript-Typdefinitionen besitzen.

Durch die Verwendung des Typs `any` teilen Sie dem TypeScript-Compiler mit, dass Werte ohne Einschränkungen dargestellt werden sollen. Beachten Sie Folgendes, um die Typsicherheit in Ihrem Code zu maximieren:

- Beschränken Sie die Verwendung von `any` auf bestimmte Fälle, in denen der Typ wirklich unbekannt ist.
- Geben Sie aus einer Funktion keine `any`-Typen zurück, da dies die Typsicherheit im Code schwächt, der sie verwendet.
- Verwenden Sie anstelle von `any` `@ts-ignore`, wenn Sie den Compiler stummschalten müssen.

```ts
let value: any;
value = true; // Valid
value = 7; // Valid
```

Typ unknown

In TypeScript stellt der Typ `unknown` einen Wert unbekannten Typs dar. Im Gegensatz zum Typ `any`, der jeden Werttyp zulässt, erfordert `unknown` eine Typprüfung oder Typassertion, bevor er auf bestimmte Weise verwendet werden kann. Daher sind für einen Wert vom Typ `unknown` keine Operationen zulässig, ohne dass er zuvor zugesichert oder auf einen spezifischeren Typ eingegrenzt wurde.

Der Typ `unknown` kann nur `any` und `unknown` selbst zugewiesen werden und ist eine typsichere Alternative zu `any`.

```
let value: unknown;

let value1: unknown = value; // Valid
let value2: any = value; // Valid
let value3: boolean = value; // Invalid
let value4: number = value; // Invalid

const add = (a: unknown, b: unknown): number | undefined =>
     typeof a === 'number' && typeof b === 'number' ? a + b :
undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Typ void

Der Typ `void` wird verwendet, um anzugeben, dass eine Funktion keinen Wert zurückgibt.

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

Typ never

Der Typ `never` stellt Werte dar, die niemals auftreten. Er wird verwendet, um Funktionen oder Ausdrücke zu kennzeichnen, die entweder niemals zurückkehren oder einen Fehler auslösen.

Zum Beispiel eine Endlosschleife:

```
const infiniteLoop = (): never => {
    while (true) {
        // do something
    }
};
```

Auslösen eines Fehlers:

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

Der Typ `never` ist nützlich, um Typsicherheit zu gewährleisten und potenzielle Fehler in Ihrem Code zu erkennen. In Kombination mit anderen Typen und Kontrollflussanweisungen hilft er TypeScript dabei, präzisere Typen zu analysieren und abzuleiten, zum Beispiel:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // move up
            break;
        case 'down':
            // move down
            break;
        default:
            const exhaustiveCheck: never = direction;
```

```
                    throw new Error(`Unhandled direction:
${exhaustiveCheck}`);
    }
};
```

Interface und Typ

Allgemeine Syntax

In TypeScript definieren Interfaces die Struktur von Objekten. Sie geben die Namen und Typen der Eigenschaften oder Methoden an, die ein Objekt besitzen muss. Die allgemeine Syntax zum Definieren eines Interfaces in TypeScript lautet wie folgt:

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}
```

Ähnlich verhält es sich bei der Definition eines Typs:

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

`interface InterfaceName` oder `type TypeName`: Definiert den Namen des Interfaces. `property1: Type1`: Gibt die Eigenschaften des Interfaces zusammen mit ihren jeweiligen Typen an. Es können mehrere Eigenschaften definiert werden, die jeweils durch ein Semikolon getrennt sind. `method1(arg1: ArgType1,`

`arg2: ArgType2): ReturnType;`: Gibt die Methoden des Interfaces an. Methoden werden mit ihrem Namen definiert, gefolgt von einer Parameterliste in Klammern und dem Rückgabetyp. Es können mehrere Methoden definiert werden, die jeweils durch ein Semikolon getrennt sind.

Beispiel für ein Interface:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

Beispiel für einen Typ:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

In TypeScript werden Typen verwendet, um die Form von Daten zu definieren und die Typprüfung durchzusetzen. Abhängig vom jeweiligen Anwendungsfall gibt es mehrere gebräuchliche Syntaxvarianten zum Definieren von Typen in TypeScript. Hier sind einige Beispiele:

Grundlegende Typen

```
let myNumber: number = 123; // number type
let myBoolean: boolean = true; // boolean type
let myArray: string[] = ['a', 'b']; // array of strings
let myTuple: [string, number] = ['a', 123]; // tuple
```

Objekte und Interfaces

```
const x: { name: string; age: number } = { name: 'Simon', age:
7 };
```

Union- und Intersection-Typen

```
type MyType = string | number; // Union type
let myUnion: MyType = 'hello'; // Can be a string
myUnion = 123; // Or a number

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Intersection type
let myCombined: CombinedType = { name: 'John', age: 25 }; //
Object with both name and age properties
```

Eingebaute primitive Typen

TypeScript verfügt über mehrere eingebaute primitive Typen, mit denen Variablen, Funktionsparameter und Rückgabetypen definiert werden können:

- `number`: Stellt numerische Werte dar, einschließlich Ganzzahlen und Gleitkommazahlen.
- `string`: Stellt Textdaten dar
- `boolean`: Stellt logische Werte dar, die entweder true oder false sein können.
- `null`: Stellt das Fehlen eines Werts dar.
- `undefined`: Stellt einen Wert dar, der nicht zugewiesen oder nicht definiert wurde.
- `symbol`: Stellt einen eindeutigen Bezeichner dar. Symbole werden üblicherweise als Schlüssel für Objekteigenschaften verwendet.
- `bigint`: Stellt Ganzzahlen beliebiger Genauigkeit dar.

- `any`: Stellt einen dynamischen oder unbekannten Typ dar. Variablen vom Typ any können Werte jedes Typs enthalten und umgehen die Typprüfung.
- `void`: Stellt das Fehlen eines beliebigen Typs dar. Er wird üblicherweise als Rückgabetyp von Funktionen verwendet, die keinen Wert zurückgeben.
- `never`: Stellt einen Typ für Werte dar, die niemals auftreten. Er wird üblicherweise als Rückgabetyp von Funktionen verwendet, die einen Fehler auslösen oder in eine Endlosschleife eintreten.

Häufig verwendete eingebaute JS-Objekte

TypeScript ist eine Obermenge von JavaScript und enthält alle häufig verwendeten eingebauten JavaScript-Objekte. Eine umfangreiche Liste dieser Objekte finden Sie auf der Dokumentationswebsite des Mozilla Developer Network (MDN): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Hier ist eine Liste einiger häufig verwendeter eingebauter JavaScript-Objekte:

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String

- RegExp
- Array
- Map
- Set
- Promise
- Intl

Überladungen

Funktionsüberladungen in TypeScript ermöglichen es Ihnen, mehrere Funktionssignaturen für einen einzelnen Funktionsnamen zu definieren. So können Sie Funktionen definieren, die auf unterschiedliche Weise aufgerufen werden können. Hier ist ein Beispiel:

```
// Overloads
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// Implementation
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // Valid
sayHi(['aa', 'bb']); // Valid
```

Hier ist ein weiteres Beispiel für die Verwendung von Funktionsüberladungen innerhalb einer `class`:

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }

    // overload
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // implementation
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
                    return  name.map(name  =>  `${this.message},
${name}!`);
        }
        throw new Error('value is invalid');
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

Zusammenführung und Erweiterung

Zusammenführung und Erweiterung beziehen sich auf zwei unterschiedliche Konzepte bei der Arbeit mit Typen und Interfaces.

Durch Zusammenführung können Sie mehrere Deklarationen mit demselben Namen zu einer einzigen Definition kombinieren, beispielsweise wenn Sie ein Interface mit demselben Namen mehrfach definieren:

```
interface X {
    a: string;
```

```
}

interface X {
    b: number;
}

const person: X = {
    a: 'a',
    b: 7,
};
```

Erweiterung bezeichnet die Möglichkeit, bestehende Typen oder Interfaces zu erweitern oder von ihnen zu erben, um neue zu erstellen. Sie ist ein Mechanismus, mit dem zusätzliche Eigenschaften oder Methoden zu einem bestehenden Typ hinzugefügt werden, ohne dessen ursprüngliche Definition zu ändern. Beispiel:

```
interface Animal {
    name: string;
    eat(): void;
}

interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

Unterschiede zwischen Typ und Interface

Deklarationszusammenführung (Augmentation):

Interfaces unterstützen die Deklarationszusammenführung. Das bedeutet, dass Sie mehrere Interfaces mit demselben Namen definieren können und TypeScript sie zu einem einzigen Interface mit den kombinierten Eigenschaften und Methoden zusammenführt. Typen hingegen unterstützen keine Deklarationszusammenführung. Dies kann hilfreich sein, wenn Sie zusätzliche Funktionalität hinzufügen oder bestehende Typen anpassen möchten, ohne die ursprünglichen Definitionen zu ändern oder fehlende beziehungsweise fehlerhafte Typen zu korrigieren.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};
```

Erweitern anderer Typen/Interfaces:

Sowohl Typen als auch Interfaces können andere Typen/Interfaces erweitern, die Syntax ist jedoch unterschiedlich. Bei Interfaces verwenden Sie das Schlüsselwort `extends`, um Eigenschaften und Methoden von anderen Interfaces zu erben. Ein Interface kann jedoch keinen komplexen Typ wie einen Union-Typ erweitern.

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
    y: 123,
    z: 'z',
};
```

Bei Typen verwenden Sie den Operator &, um mehrere Typen zu einem einzigen Typ zu kombinieren (Intersection).

```
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
    y: 123,
    j: 'j',
};
```

Union- und Intersection-Typen:

Typen sind bei der Definition von Union- und Intersection-Typen flexibler. Mit dem Schlüsselwort `type` können Sie Union-Typen einfach mithilfe des Operators `|` und Intersection-Typen mithilfe des Operators `&` erstellen. Interfaces können Union-

Typen zwar ebenfalls indirekt darstellen, verfügen jedoch nicht über eine integrierte Unterstützung für Intersection-Typen.

```
type Department = 'dep-x' | 'dep-y'; // Union

type Person = {
    name: string;
    age: number;
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // Intersection
```

Beispiel mit Interfaces:

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // Union of interfaces
```

Klasse

Allgemeine Syntax einer Klasse

Das Schlüsselwort `class` wird in TypeScript verwendet, um eine Klasse zu definieren. Unten sehen Sie ein Beispiel:

```
class Person {
    private name: string;
```

```
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
```

Das Schlüsselwort `class` wird verwendet, um eine Klasse namens “Person” zu definieren.

Die Klasse besitzt zwei private Eigenschaften: name vom Typ `string` und age vom Typ `number`.

Der Konstruktor wird mit dem Schlüsselwort `constructor` definiert. Er nimmt name und age als Parameter entgegen und weist sie den entsprechenden Eigenschaften zu.

Die Klasse besitzt eine `public`-Methode namens sayHi, die eine Begrüßungsnachricht protokolliert.

Um in TypeScript eine Instanz einer Klasse zu erstellen, können Sie das Schlüsselwort `new` verwenden, gefolgt vom Klassennamen und den Klammern `()`. Zum Beispiel:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Output: Hello, my name is John Doe and I
am 25 years old.
```

Konstruktor

Konstruktoren sind spezielle Methoden innerhalb einer Klasse, mit denen die Eigenschaften des Objekts initialisiert werden, wenn eine Instanz der Klasse erstellt wird.

```
class Person {
    public name: string;
    public age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
            `Hello, my name is ${this.name} and I'm ${this.age}
years old.`
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

Ein Konstruktor kann mit der folgenden Syntax überladen werden:

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
```

```
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

In TypeScript können mehrere Konstruktorüberladungen definiert werden, es kann jedoch nur eine Implementierung geben, die mit allen Überladungen kompatibel sein muss. Dies lässt sich durch die Verwendung eines optionalen Parameters erreichen.

```
class Person {
    name: string;
    age: number;

    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0
```

```
const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

Private und geschützte Konstruktoren

In TypeScript können Konstruktoren mit private oder protected gekennzeichnet werden, wodurch ihre Zugänglichkeit und Verwendung eingeschränkt wird.

Private Konstruktoren: Sie können nur innerhalb der Klasse selbst aufgerufen werden. Private Konstruktoren werden häufig in Szenarien verwendet, in denen ein Singleton-Muster erzwungen oder die Erstellung von Instanzen auf eine Factory-Methode innerhalb der Klasse beschränkt werden soll.

Geschützte Konstruktoren: Geschützte Konstruktoren sind nützlich, wenn Sie eine Basisklasse erstellen möchten, die nicht direkt instanziiert werden soll, aber von Unterklassen erweitert werden kann.

```
class BaseClass {
    protected constructor() {}
}

class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// Attempting to instantiate the base class directly will
result in an error
```

```
// const baseObj = new BaseClass(); // Error: Constructor of
class 'BaseClass' is protected.

// Create an instance of the derived class
const derivedObj = new DerivedClass(10);
```

Zugriffsmodifikatoren

Die Zugriffsmodifikatoren `private`, `protected` und `public` werden verwendet, um die Sichtbarkeit und Zugänglichkeit von Klassenmitgliedern wie Eigenschaften und Methoden in TypeScript-Klassen zu steuern. Diese Modifikatoren sind entscheidend, um Kapselung durchzusetzen und Grenzen für den Zugriff auf den internen Zustand einer Klasse sowie dessen Änderung festzulegen.

Der Modifikator `private` beschränkt den Zugriff auf das Klassenmitglied auf die enthaltende Klasse.

Der Modifikator `protected` erlaubt den Zugriff auf das Klassenmitglied innerhalb der enthaltenden Klasse und ihrer abgeleiteten Klassen.

Der Modifikator `public` ermöglicht uneingeschränkten Zugriff auf das Klassenmitglied, sodass von überall darauf zugegriffen werden kann.

Get und Set

Getter und Setter sind spezielle Methoden, mit denen Sie benutzerdefiniertes Verhalten für den Zugriff auf Klasseneigenschaften und deren Änderung definieren können. Sie ermöglichen es Ihnen, den internen Zustand eines Objekts zu kapseln und beim Abrufen oder Festlegen von

Eigenschaftswerten zusätzliche Logik bereitzustellen. In TypeScript werden Getter beziehungsweise Setter mit den Schlüsselwörtern `get` und `set` definiert. Hier ist ein Beispiel:

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

Auto-Accessors in Klassen

TypeScript-Version 4.9 fügt Unterstützung für Auto-Accessors hinzu, eine kommende ECMAScript-Funktion. Sie ähneln Klasseneigenschaften, werden jedoch mit dem Schlüsselwort “accessor” deklariert.

```
class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```

Auto-Accessors werden in private `get`- und `set`-Accessors aufgelöst, die mit einer unzugänglichen Eigenschaft arbeiten.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
    }
}
```

this

In TypeScript bezieht sich das Schlüsselwort `this` innerhalb der Methoden oder Konstruktoren einer Klasse auf deren aktuelle Instanz. Es ermöglicht Ihnen, innerhalb des eigenen Gültigkeitsbereichs der Klasse auf deren Eigenschaften und Methoden zuzugreifen und sie zu ändern. Es bietet eine Möglichkeit, innerhalb der eigenen Methoden eines Objekts auf dessen internen Zustand zuzugreifen und ihn zu bearbeiten.

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

Parametereigenschaften

Mit Parametereigenschaften können Sie Klasseneigenschaften direkt in den Konstruktorparametern deklarieren und initialisieren und so Boilerplate-Code vermeiden. Zum Beispiel:

```
class Person {
    constructor(
        private name: string,
        public age: number
    ) {
            // The "private" and "public" keywords in the
constructor
              // automatically declare and initialize the
corresponding class properties.
    }
    public introduce(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();
```

Abstrakte Klassen

Abstrakte Klassen werden in TypeScript hauptsächlich für Vererbung verwendet. Sie bieten eine Möglichkeit, gemeinsame Eigenschaften und Methoden zu definieren, die von Unterklassen geerbt werden können. Dies ist nützlich, wenn Sie gemeinsames Verhalten definieren und erzwingen möchten, dass Unterklassen bestimmte Methoden implementieren. Sie ermöglichen es, eine Klassenhierarchie zu erstellen, in der die

abstrakte Basisklasse ein gemeinsames Interface und gemeinsame Funktionalität für die Unterklassen bereitstellt.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Output: Whiskers meows.
```

Mit Generics

Klassen mit Generics ermöglichen es Ihnen, wiederverwendbare Klassen zu definieren, die mit verschiedenen Typen arbeiten können.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
```

```
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

Decorators

Decorators bieten einen Mechanismus, um Metadaten hinzuzufügen, Verhalten zu ändern, Validierungen durchzuführen oder die Funktionalität des Zielelements zu erweitern. Sie sind Funktionen, die zur Laufzeit ausgeführt werden. Auf eine Deklaration können mehrere Decorators angewendet werden.

Decorators sind experimentelle Funktionen, und die folgenden Beispiele sind nur mit TypeScript-Version 5 oder höher unter Verwendung von ES6 kompatibel.

Für TypeScript-Versionen vor 5 sollten sie über die Eigenschaft `experimentalDecorators` in Ihrer `tsconfig.json` oder durch die Verwendung von `--experimentalDecorators` in Ihrer Befehlszeile aktiviert werden (das folgende Beispiel funktioniert jedoch nicht).

Zu den häufigen Anwendungsfällen für Decorators gehören:

- Überwachen von Eigenschaftsänderungen.
- Überwachen von Methodenaufrufen.
- Hinzufügen zusätzlicher Eigenschaften oder Methoden.
- Validierung zur Laufzeit.
- Automatische Serialisierung und Deserialisierung.
- Protokollierung.
- Autorisierung und Authentifizierung.
- Absicherung gegen Fehler.

Hinweis: Decorators für Version 5 erlauben es nicht, Parameter zu dekorieren.

Arten von Decorators:

Klassen-Decorators

Klassen-Decorators sind nützlich, um eine bestehende Klasse zu erweitern, etwa durch das Hinzufügen von Eigenschaften oder Methoden oder das Sammeln von Instanzen einer Klasse. Im folgenden Beispiel fügen wir eine `toString`-Methode hinzu, die die Klasse in eine Zeichenfolgendarstellung umwandelt.

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
```

```
}

@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* Logs:
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

Eigenschafts-Decorator

Eigenschafts-Decorators sind nützlich, um das Verhalten einer Eigenschaft zu ändern, beispielsweise deren Initialisierungswerte. Im folgenden Code haben wir ein Skript, das eine Eigenschaft so festlegt, dass sie immer in Großbuchstaben vorliegt:

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}
```

```
class MyClass {
    @upperCase
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // Logs: HELLO!
```

Methoden-Decorator

Mit Methoden-Decorators können Sie das Verhalten von Methoden ändern oder erweitern. Unten sehen Sie ein Beispiel für einen einfachen Logger:

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

       function replacementMethod(this: This, ...args: Args):
Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
```

```
    }
}

new MyClass().sayHello();
```

Die Ausgabe lautet:

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

Getter- und Setter-Decorators

Mit Getter- und Setter-Decorators können Sie das Verhalten von Klassen-Accessors ändern oder erweitern. Sie sind beispielsweise nützlich, um Zuweisungen an Eigenschaften zu validieren. Hier ist ein einfaches Beispiel für einen Getter-Decorator:

```
function range<This, Return extends number>(min: number, max:
number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Invalid';
            }
            Object.defineProperty(this, context.name, {
                value,
                enumerable: true,
            });
            return value;
        };
    };
```

```
}

class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Valid: 10

const obj2 = new MyClass(999);
console.log(obj2.getValue); // Throw: Invalid!
```

Decorator-Metadaten

Decorator-Metadaten erleichtern es Decorators, Metadaten in einer beliebigen Klasse anzuwenden und zu verwenden. Decorators können auf eine neue Metadateneigenschaft des Kontextobjekts zugreifen, die sowohl für primitive Werte als auch für Objekte als Schlüssel dienen kann. Auf Metadateninformationen kann in der Klasse über `Symbol.metadata` zugegriffen werden.

Metadaten können für verschiedene Zwecke verwendet werden, etwa für Debugging, Serialisierung oder Dependency Injection mit Decorators.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Simple
polyfill

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
    | ClassMethodDecoratorContext; // Context contains property
metadata: DecoratorMetadata

function setMetadata(_target: any, context: Context) {
    // Set the metadata object with a primitive value
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;

    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Get metadata
information

console.log(JSON.stringify(metadata));                    //
{"bar":true,"baz":true,"foo":true}
```

Vererbung

Vererbung bezeichnet den Mechanismus, durch den eine Klasse Eigenschaften und Methoden von einer anderen Klasse erben kann, die als Basisklasse oder Oberklasse bezeichnet wird. Die

abgeleitete Klasse, auch Kindklasse oder Unterklasse genannt, kann die Funktionalität der Basisklasse erweitern und spezialisieren, indem sie neue Eigenschaften und Methoden hinzufügt oder bestehende überschreibt.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Create an instance of the base class
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// Create an instance of the derived class
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript unterstützt keine Mehrfachvererbung im traditionellen Sinn, sondern erlaubt stattdessen die Vererbung von einer einzigen Basisklasse. TypeScript unterstützt mehrere Interfaces. Ein Interface kann einen Vertrag für die Struktur eines Objekts definieren, und eine Klasse kann mehrere Interfaces implementieren. Dadurch kann eine Klasse Verhalten und Struktur aus mehreren Quellen erben.

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

Das Schlüsselwort `class` in TypeScript wird, ähnlich wie in JavaScript, häufig als syntaktischer Zucker bezeichnet. Es wurde in ECMAScript 2015 (ES6) eingeführt, um eine vertrautere Syntax für das klassenbasierte Erstellen und Verwenden von Objekten bereitzustellen. Dabei ist jedoch zu beachten, dass TypeScript als Obermenge von JavaScript

letztlich zu JavaScript kompiliert wird, das im Kern weiterhin prototypbasiert ist.

Statische Member

TypeScript verfügt über statische Member. Um auf die statischen Member einer Klasse zuzugreifen, können Sie den Klassennamen gefolgt von einem Punkt verwenden, ohne ein Objekt erstellen zu müssen.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

Initialisierung von Eigenschaften

Es gibt mehrere Möglichkeiten, Eigenschaften einer Klasse in TypeScript zu initialisieren:

Inline:

Im folgenden Beispiel werden diese Anfangswerte verwendet, wenn eine Instanz der Klasse erstellt wird.

```
class MyClass {
    property1: string = 'default value';
```

```
    property2: number = 42;
}
```

Im Konstruktor:

```
class MyClass {
    property1: string;
    property2: number;

    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

Mit Konstruktorparametern:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
        public property2: number = 42
    ) {
            // There is no need to assign the values to the
properties explicitly.
    }
    log() {
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

Methodenüberladung

Die Methodenüberladung ermöglicht es einer Klasse, mehrere Methoden mit demselben Namen, aber unterschiedlichen Parametertypen oder einer unterschiedlichen Anzahl von Parametern zu besitzen. Dadurch können wir eine Methode

abhängig von den übergebenen Argumenten auf unterschiedliche Weise aufrufen.

```
class MyClass {
    add(a: number, b: number): number; // Overload signature 1
    add(a: string, b: string): string; // Overload signature 2

      add(a: number | string, b: number | string): number |
string {
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
}

const r = new MyClass();
console.log(r.add(10, 5)); // Logs 15
```

Generics

Mit Generics können Sie wiederverwendbare Komponenten und Funktionen erstellen, die mit mehreren Typen arbeiten können. Mithilfe von Generics können Sie Typen, Funktionen und Interfaces parametrisieren, sodass sie mit verschiedenen Typen arbeiten können, ohne diese vorher explizit anzugeben.

Mit Generics können Sie Code flexibler und wiederverwendbarer gestalten.

Generischer Typ

Um einen generischen Typ zu definieren, verwenden Sie spitze Klammern (<>) zur Angabe der Typparameter, zum Beispiel:

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

Generische Klassen

Generics können auch auf Klassen angewendet werden. Auf diese Weise können diese mithilfe von Typparametern mit mehreren Typen arbeiten. Dies ist nützlich, um wiederverwendbare Klassendefinitionen zu erstellen, die mit verschiedenen Datentypen arbeiten und dabei die Typsicherheit gewährleisten.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123
```

```
const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

Einschränkungen für Generics

Generische Parameter können mit dem Schlüsselwort `extends` eingeschränkt werden, gefolgt von einem Typ oder Interface, dem der Typparameter entsprechen muss.

Im folgenden Beispiel muss `T` über eine korrekt typisierte Eigenschaft `length` verfügen, um gültig zu sein:

```
const printLen = <T extends { length: number }>(value: T): void
=> {
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // Invalid
```

Eine bemerkenswerte Funktion von Generics, die in Version 3.4 RC eingeführt wurde, ist die Typinferenz für Funktionen höherer Ordnung, die generische Typargumente weitergibt:

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

Diese Funktionalität erleichtert die typsichere Programmierung im Point-free-Stil, die in der funktionalen Programmierung üblich ist.

Kontextbezogenes Narrowing für Generics

Kontextbezogenes Narrowing für Generics ist der Mechanismus in TypeScript, der es dem Compiler ermöglicht, den Typ eines generischen Parameters anhand des Kontexts einzugrenzen, in dem er verwendet wird. Es ist bei der Arbeit mit generischen Typen in bedingten Anweisungen nützlich:

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // Value is narrowed down to type 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // Value is narrowed down to type 'number'
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

Strukturelle Typen mit Typlöschung

In TypeScript müssen Objekte keinem bestimmten, exakten Typ entsprechen. Wenn wir beispielsweise ein Objekt erstellen, das die Anforderungen eines Interfaces erfüllt, können wir dieses Objekt an Stellen verwenden, an denen dieses Interface erforderlich ist, selbst wenn zwischen beiden keine explizite Verbindung besteht. Beispiel:

```ts
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
    prop1: 'Origin',
};

log(obj); // Valid
```

Namespaces

In TypeScript werden Namespaces verwendet, um Code in logischen Containern zu organisieren, Namenskollisionen zu vermeiden und zusammengehörigen Code zu gruppieren. Die Verwendung des Schlüsselworts `export` ermöglicht den Zugriff auf den Namespace von außerhalb von Modulen.

```ts
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

Symbole

Symbole sind ein primitiver Datentyp, der einen unveränderlichen Wert darstellt, dessen globale Eindeutigkeit während der gesamten Laufzeit des Programms garantiert ist.

Symbole können als Schlüssel für Objekteigenschaften verwendet werden und bieten eine Möglichkeit, nicht aufzählbare Eigenschaften zu erstellen.

```
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

In WeakMaps und WeakSets sind Symbole nun als Schlüssel zulässig.

Triple-Slash-Direktiven

Triple-Slash-Direktiven sind spezielle Kommentare, die dem Compiler Anweisungen dazu geben, wie eine Datei verarbeitet werden soll. Diese Direktiven beginnen mit drei aufeinanderfolgenden Schrägstrichen (`///`), stehen üblicherweise am Anfang einer TypeScript-Datei und haben keine Auswirkungen auf das Laufzeitverhalten.

Triple-Slash-Direktiven werden verwendet, um auf externe Abhängigkeiten zu verweisen, das Verhalten beim Laden von Modulen festzulegen, bestimmte Compilerfunktionen zu aktivieren oder zu deaktivieren und vieles mehr. Einige Beispiele:

Verweisen auf eine Deklarationsdatei:

```
/// <reference path="path/to/declaration/file.d.ts" />
```

Angeben des Modulformats:

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

Aktivieren von Compileroptionen, im folgenden Beispiel der Strict-Modus:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

Typmanipulation

Erstellen von Typen aus Typen

Neue Typen können durch das Kombinieren, Manipulieren oder Transformieren vorhandener Typen erstellt werden.

Intersection Types (`&`):

Mit ihnen können Sie mehrere Typen zu einem einzigen Typ kombinieren:

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Intersection of A and B
const obj: C = { foo: 42, bar: 'hello' };
```

Union Types (|):

Mit ihnen können Sie einen Typ definieren, der einem von mehreren Typen entsprechen kann:

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

Mapped Types:

Mit ihnen können Sie die Eigenschaften eines vorhandenen Typs transformieren, um einen neuen Typ zu erstellen:

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // Properties become read-only
```

Conditional Types:

Mit ihnen können Sie Typen auf der Grundlage bestimmter Bedingungen erstellen:

```
type ExtractParam<T> = T extends (param: infer P) => any ? P : never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

Indexed Access Types

In TypeScript können Sie mithilfe eines Indexes, `Type[Key]`, auf die Typen von Eigenschaften innerhalb eines anderen Typs zugreifen und sie manipulieren.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

Utility Types

Zur Manipulation von Typen stehen mehrere integrierte Utility Types zur Verfügung. Nachfolgend finden Sie eine Liste der am häufigsten verwendeten:

Awaited<T>

Erstellt einen Typ, der Promise-Typen rekursiv entpackt.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

Erstellt einen Typ, bei dem alle Eigenschaften von T optional sind.

```
type Person = {
    name: string;
    age: number;
};
```

```ts
type A = Partial<Person>; // { name?: string | undefined; age?: number | undefined; }
```

Required<T>

Erstellt einen Typ, bei dem alle Eigenschaften von T erforderlich sind.

```ts
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

Erstellt einen Typ, bei dem alle Eigenschaften von T schreibgeschützt sind.

```ts
type Person = {
    name: string;
    age: number;
};

type A = Readonly<Person>;

const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // Invalid
```

Record<K, T>

Erstellt einen Typ mit einer Menge von Eigenschaften K des Typs T.

```ts
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

Erstellt einen Typ, indem die angegebenen Eigenschaften K aus T ausgewählt werden.

```ts
type Product = {
    name: string;
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

Erstellt einen Typ, indem die angegebenen Eigenschaften K aus T weggelassen werden.

```ts
type Product = {
    name: string;
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

Erstellt einen Typ, indem alle Werte des Typs U aus T ausgeschlossen werden.

```
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

Erstellt einen Typ, indem alle Werte des Typs U aus T extrahiert werden.

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

Erstellt einen Typ, indem null und undefined aus T ausgeschlossen werden.

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

Extrahiert die Parametertypen eines Funktionstyps T.

```
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

Extrahiert die Parametertypen eines Konstruktorfunktionstyps T.

```
class Person {
    constructor(
        public name: string,
        public age: number
    ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

ReturnType<T>

Extrahiert den Rückgabetyp eines Funktionstyps T.

```
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

Extrahiert den Instanztyp eines Klassentyps T.

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}
```

```
type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

Extrahiert den Typ des ‘this’-Parameters aus einem Funktionstyp T.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type PersonThisType = ThisParameterType<Person['greet']>; // Person
```

OmitThisParameter<T>

Entfernt den ‘this’-Parameter aus einem Funktionstyp T.

```
function capitalize(this: String) {
                        return      this[0].toUpperCase()      + this.substring(1).toLowerCase();
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; // () => string
```

ThisType<T>

Dient als Marker für einen kontextbezogenen `this`-Typ.

```
type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } &
ThisType<Logger> = {
    hello: function () {
        this.log('some error'); // Valid as "log" is a part of
"this".
        this.update(); // Invalid
    },
};
```

Uppercase<T>

Wandelt den Namen des Eingabetyps T in Großbuchstaben um.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

Wandelt den Namen des Eingabetyps T in Kleinbuchstaben um.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

Schreibt den Namen des Eingabetyps T am Anfang groß.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

Schreibt den Namen des Eingabetyps T am Anfang klein.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer ist ein Utility Type, der die automatische Typinferenz innerhalb des Gültigkeitsbereichs einer generischen Funktion verhindert.

Beispiel:

```
// Automatic inference of types within the scope of a generic function.
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Type here is ("a" | "b" | "c")[]
```

Mit NoInfer:

```
// Example function that uses NoInfer to prevent type inference
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // Error: Type Argument of type '"c"' is not assignable to parameter of type '"a" | "b"'.
```

Sonstiges

Fehler- und Ausnahmebehandlung

TypeScript ermöglicht es Ihnen, Fehler mit den standardmäßigen Fehlerbehandlungsmechanismen von JavaScript abzufangen und zu behandeln:

Try-Catch-Finally-Blöcke:

```
try {
    // Code that might throw an error
} catch (error) {
    // Handle the error
} finally {
    // Code that always executes, finally is optional
}
```

Sie können auch verschiedene Fehlertypen behandeln:

```
try {
    // Code that might throw different types of errors
} catch (error) {
    if (error instanceof TypeError) {
        // Handle TypeError
    } else if (error instanceof RangeError) {
        // Handle RangeError
    } else {
        // Handle other errors
    }
}
```

Benutzerdefinierte Fehlertypen:

Durch Erweitern der `Error`-Klasse können spezifischere Fehler definiert werden:

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'CustomError';
    }
}

throw new CustomError('This is a custom error.');
```

Mixin-Klassen

Mixin-Klassen ermöglichen es Ihnen, das Verhalten mehrerer Klassen in einer einzigen Klasse zu kombinieren und zusammenzustellen. Sie bieten eine Möglichkeit, Funktionalität wiederzuverwenden und zu erweitern, ohne tiefe Vererbungsketten zu benötigen.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// Extend MyClass to include the behavior of Identifiable and
Selectable
interface MyClass extends Identifiable, Selectable {}

// Function to apply mixins to a class
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {

Object.getOwnPropertyNames(baseCtor.prototype).forEach(name   =>
{
            let descriptor = Object.getOwnPropertyDescriptor(
```

```
                baseCtor.prototype,
                name
            );
            if (descriptor) {
                Object.defineProperty(source.prototype, name,
descriptor);
            }
        });
    });
}

// Apply the mixins to MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

Asynchrone Sprachfunktionen

Da TypeScript eine Obermenge von JavaScript ist, verfügt es über die integrierten asynchronen Sprachfunktionen von JavaScript, darunter:

Promises:

Promises bieten eine Möglichkeit, asynchrone Operationen und deren Ergebnisse zu verarbeiten. Dabei werden Methoden wie `.then()` und `.catch()` verwendet, um Erfolgs- und Fehlerfälle zu behandeln.

Weitere Informationen: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

Die Schlüsselwörter `async` und `await` ermöglichen bei der Arbeit mit Promises eine Syntax, die eher wie synchroner Code aussieht. Mit dem Schlüsselwort `async` wird eine asynchrone Funktion definiert. Das Schlüsselwort `await` wird innerhalb einer asynchronen Funktion verwendet, um die Ausführung anzuhalten, bis ein Promise erfüllt oder abgelehnt wird.

Weitere Informationen: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

Die folgenden APIs werden in TypeScript umfassend unterstützt:

Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Workers: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Workers: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Iteratoren und Generatoren

Sowohl Iteratoren als auch Generatoren werden in TypeScript umfassend unterstützt.

Iteratoren sind Objekte, die das Iterator-Protokoll implementieren und den schrittweisen Zugriff auf die Elemente

einer Sammlung oder Sequenz ermöglichen. Ein Iterator ist eine Struktur, die einen Zeiger auf das nächste Element der Iteration enthält. Iteratoren besitzen eine `next()`-Methode, die den nächsten Wert der Sequenz zusammen mit einem booleschen Wert zurückgibt, der angibt, ob die Sequenz `done` ist.

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

Generatoren sind spezielle Funktionen, die mit der `function*`-Syntax definiert werden und das Erstellen von Iteratoren vereinfachen. Sie verwenden das Schlüsselwort `yield`, um die Folge von Werten zu definieren, und halten die Ausführung automatisch an beziehungsweise setzen sie fort, wenn Werte angefordert werden.

Generatoren erleichtern das Erstellen von Iteratoren und sind besonders bei der Arbeit mit großen oder unendlichen Sequenzen nützlich.

Beispiel:

```
function* numberGenerator(start: number, end: number): Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript unterstützt außerdem asynchrone Iteratoren und asynchrone Generatoren.

Weitere Informationen:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

TsDocs-JSDoc-Referenz

Bei der Arbeit mit einer JavaScript-Codebasis können Sie TypeScript durch JSDoc-Kommentare mit zusätzlichen Annotationen Typinformationen bereitstellen und so bei der korrekten Typinferenz unterstützen.

Beispiel:

```
/**
 * Computes the power of a given number
 * @constructor
 * @param {number} base - The base value of the expression
  * @param {number} exponent - The exponent value of the expression
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number): number
```

Die vollständige Dokumentation finden Sie unter diesem Link: https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

Ab Version 3.7 können .d.ts-Typdefinitionen aus der JavaScript-JSDoc-Syntax generiert werden. Weitere Informationen finden Sie hier: https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

@types

Pakete unter der @types-Organisation folgen einer speziellen Namenskonvention und stellen Typdefinitionen für vorhandene JavaScript-Bibliotheken oder -Module bereit. Beispielsweise wird durch:

```
npm install --save-dev @types/lodash
```

die Typdefinition von `lodash` in Ihrem aktuellen Projekt installiert.

Wenn Sie zu den Typdefinitionen eines `@types`-Pakets beitragen möchten, reichen Sie bitte einen Pull Request unter https://github.com/DefinitelyTyped/DefinitelyTyped ein.

JSX

JSX (JavaScript XML) ist eine Erweiterung der JavaScript-Syntax, mit der Sie HTML-ähnlichen Code in Ihren JavaScript- oder TypeScript-Dateien schreiben können. JSX wird häufig in React verwendet, um die HTML-Struktur zu definieren.

TypeScript erweitert die Möglichkeiten von JSX durch Typprüfung und statische Analyse.

Um JSX zu verwenden, müssen Sie die Compileroption `jsx` in Ihrer Datei `tsconfig.json` festlegen. Zwei häufig verwendete Konfigurationsoptionen sind:

- “preserve”: Gibt .jsx-Dateien mit unverändertem JSX aus. Diese Option weist TypeScript an, die JSX-Syntax unverändert beizubehalten und sie während des Kompilierungsvorgangs nicht zu transformieren. Sie

können diese Option verwenden, wenn ein separates Tool wie Babel die Transformation übernimmt.
- "react": Aktiviert die integrierte JSX-Transformation von TypeScript. React.createElement wird verwendet.

Alle Optionen finden Sie hier: https://www.typescriptlang.org/tsconfig#jsx

ES6-Module

TypeScript unterstützt ES6 (ECMAScript 2015) und viele nachfolgende Versionen. Das bedeutet, dass Sie ES6-Syntax wie Arrow Functions, Template Literals, Klassen, Module, Destrukturierung und mehr verwenden können.

Um ES6-Features in Ihrem Projekt zu aktivieren, können Sie die Eigenschaft `target` in der tsconfig.json angeben.

Ein Konfigurationsbeispiel:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

ES7-Potenzierungsoperator

Der Potenzierungsoperator (**) berechnet den Wert, der sich ergibt, wenn der erste Operand mit dem zweiten Operanden

potenziert wird. Er funktioniert ähnlich wie `Math.pow()`, kann jedoch zusätzlich BigInts als Operanden verarbeiten. TypeScript unterstützt diesen Operator vollständig, wenn `target` in Ihrer Datei tsconfig.json auf `es2016` oder höher gesetzt ist.

```
console.log(2 ** (2 ** 2)); // 16
```

Die for-await-of-Anweisung

Dies ist ein in TypeScript vollständig unterstütztes JavaScript-Feature, mit dem Sie bei der Zielversion `es2018` über asynchron iterierbare Objekte iterieren können.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

Neue target-Metaeigenschaft

Sie können die Metaeigenschaft `new.target` in TypeScript verwenden, um festzustellen, ob eine Funktion oder ein Konstruktor mit dem new-Operator aufgerufen wurde. Damit können Sie erkennen, ob ein Objekt durch einen Konstruktoraufruf erstellt wurde.

```
class Parent {
    constructor() {
```

```
            console.log(new.target);  // Logs the constructor
function used to create an instance
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}

const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

Dynamische Importausdrücke

Mit dem von TypeScript unterstützten ECMAScript-Vorschlag für dynamische Imports können Module bedingt oder bei Bedarf verzögert geladen werden.

Die Syntax für dynamische Importausdrücke in TypeScript lautet wie folgt:

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
            const widget = await import('./widget'); // Dynamic
import
        widget.render(container);
    }
}

renderWidget();
```

“tsc –watch”

Dieser Befehl startet den TypeScript-Compiler mit dem Parameter `--watch`, sodass TypeScript-Dateien bei jeder Änderung automatisch neu kompiliert werden können.

```
tsc --watch
```

Ab TypeScript-Version 4.9 basiert die Dateiüberwachung hauptsächlich auf Dateisystemereignissen. Wenn kein ereignisbasierter Watcher eingerichtet werden kann, wird automatisch auf Polling zurückgegriffen.

Non-Null-Assertion-Operator

Der Non-Null-Assertion-Operator (Postfix !), auch als Definite Assignment Assertion bezeichnet, ist ein TypeScript-Feature, mit dem Sie bestätigen können, dass eine Variable oder Eigenschaft weder null noch undefined ist, selbst wenn die statische Typanalyse von TypeScript darauf hindeutet, dass dies möglich sein könnte. Mit diesem Feature können Sie auf explizite Prüfungen verzichten.

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
    console.log(`Name is ${person!.name}`);
};
```

Deklarationen mit Standardwerten

Deklarationen mit Standardwerten werden verwendet, wenn einer Variablen oder einem Parameter ein Standardwert zugewiesen wird. Wird für diese Variable oder diesen

Parameter kein Wert angegeben, wird stattdessen der Standardwert verwendet.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

Optional Chaining

Der Optional-Chaining-Operator `?.` funktioniert beim Zugriff auf Eigenschaften oder Methoden wie der reguläre Punktoperator (`.`). Werte, die null oder undefined sind, werden jedoch abgefangen, indem der Ausdruck beendet und `undefined` zurückgegeben wird, anstatt einen Fehler auszulösen.

```
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};

console.log(person.address?.city); // undefined
```

Nullish-Coalescing-Operator

Der Nullish-Coalescing-Operator `??` gibt den Wert auf der rechten Seite zurück, wenn der Wert auf der linken Seite `null`

oder `undefined` ist. Andernfalls gibt er den Wert auf der linken Seite zurück.

```
const foo = null ?? 'foo';
console.log(foo); // foo

const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

Template Literal Types

Mit Template Literal Types können Sie Stringwerte auf Typebene manipulieren und auf der Grundlage vorhandener Typen neue Stringtypen erzeugen. Sie eignen sich dazu, aus stringbasierten Operationen ausdrucksstärkere und präzisere Typen zu erstellen.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

Funktionsüberladung

Mit Funktionsüberladung können Sie mehrere Funktionssignaturen für denselben Funktionsnamen definieren, jeweils mit unterschiedlichen Parameter- und Rückgabetypen. Wenn Sie eine überladene Funktion aufrufen, ermittelt TypeScript anhand der angegebenen Argumente die richtige Funktionssignatur:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];
```

```
function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

Rekursive Typen

Ein rekursiver Typ ist ein Typ, der auf sich selbst verweisen kann. Dies ist nützlich, um Datenstrukturen mit einer hierarchischen oder rekursiven Struktur (potenziell unendlicher Verschachtelung) zu definieren, beispielsweise verkettete Listen, Bäume und Graphen.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

Rekursive Conditional Types

In TypeScript können komplexe Typbeziehungen mithilfe von Logik und Rekursion definiert werden. Betrachten wir dies in einfachen Schritten:

Mit Conditional Types können Sie Typen auf der Grundlage boolescher Bedingungen definieren:

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

Rekursion bezeichnet eine Typdefinition, die innerhalb ihrer eigenen Definition auf sich selbst verweist:

```
type Json = string | number | boolean | null | Json[] | { [key: string]: Json };

const data: Json = {
    prop1: true,
    prop2: 'prop2',
    prop3: {
        prop4: [],
    },
};
```

Rekursive Conditional Types kombinieren bedingte Logik und Rekursion. Das bedeutet, dass eine Typdefinition durch bedingte Logik von sich selbst abhängen kann, wodurch komplexe und flexible Typbeziehungen entstehen.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 | 1 | 6
```

Unterstützung von ECMAScript-Modulen in Node

Node.js unterstützt ECMAScript-Module seit Version 15.3.0, und TypeScript unterstützt ECMAScript-Module für Node.js seit Version 4.7. Diese Unterstützung kann aktiviert werden, indem Sie in der Datei tsconfig.json die Eigenschaft `module` auf den Wert `nodenext` setzen. Hier ist ein Beispiel:

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

Node.js unterstützt zwei Dateierweiterungen für Module: `.mjs` für ES-Module und `.cjs` für CommonJS-Module. Die entsprechenden Dateierweiterungen in TypeScript sind `.mts` für ES-Module und `.cts` für CommonJS-Module. Wenn der TypeScript-Compiler diese Dateien in JavaScript transpiliert, erstellt er `.mjs`- und `.cjs`-Dateien.

Wenn Sie ES-Module in Ihrem Projekt verwenden möchten, können Sie die Eigenschaft `type` in Ihrer Datei package.json auf “module” setzen. Dadurch wird Node.js angewiesen, das Projekt als ES-Modulprojekt zu behandeln.

Darüber hinaus unterstützt TypeScript Typdeklarationen in .d.ts-Dateien. Diese Deklarationsdateien stellen Typinformationen für in TypeScript geschriebene Bibliotheken oder Module bereit, sodass andere Entwickler sie mit der Typprüfung und den Autovervollständigungsfunktionen von TypeScript verwenden können.

Assertion Functions

In TypeScript sind Assertion Functions Funktionen, die anhand ihres Rückgabewerts die Überprüfung einer bestimmten Bedingung anzeigen. In ihrer einfachsten Form prüft eine Assert-Funktion ein angegebenes Prädikat und löst einen Fehler aus, wenn das Prädikat false ergibt.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

Alternativ kann sie als Funktionsausdruck deklariert werden:

```
type AssertIsNumber = (value: unknown) => asserts value is
number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

Assertion Functions weisen Ähnlichkeiten mit Type Guards auf. Type Guards wurden ursprünglich eingeführt, um Laufzeitprüfungen durchzuführen und den Typ eines Werts innerhalb eines bestimmten Gültigkeitsbereichs sicherzustellen. Genauer gesagt ist ein Type Guard eine Funktion, die ein Typprädikat auswertet und einen booleschen Wert zurückgibt, der angibt, ob das Prädikat true oder false ist. Dies unterscheidet sich geringfügig von Assertion Functions, bei denen ein Fehler ausgelöst werden soll, anstatt false zurückzugeben, wenn das Prädikat nicht erfüllt ist.

Beispiel für einen Type Guard:

```
const isNumber = (value: unknown): value is number => typeof
value === 'number';
```

Variadische Tupeltypen

Variadische Tupeltypen sind ein mit TypeScript-Version 4.0 eingeführtes Feature. Sehen wir uns daher zunächst noch

einmal an, was ein Tupel ist:

Ein Tupeltyp ist ein Array mit einer festgelegten Länge, bei dem der Typ jedes Elements bekannt ist:

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

Der Begriff "variadisch" bezeichnet eine unbestimmte Stelligkeit (das Akzeptieren einer variablen Anzahl von Argumenten).

Ein variadisches Tupel ist ein Tupeltyp mit allen zuvor genannten Eigenschaften, dessen genaue Form jedoch noch nicht definiert ist:

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

Im vorherigen Code ist zu sehen, dass die Form des Tupels durch den übergebenen generischen Typ `T` definiert wird.

Variadische Tupel können mehrere generische Typen akzeptieren, wodurch sie sehr flexibel sind:

```
type Bar<T extends unknown[], G extends unknown[]> = [...T,
boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean,
boolean]
```

Mit den neuen variadischen Tupeln können wir Folgendes verwenden:

- Spreads in der Tupeltypsyntax können jetzt generisch sein. Dadurch können wir Operationen höherer Ordnung für Tupel und Arrays darstellen, selbst wenn wir die tatsächlichen Typen, mit denen wir arbeiten, nicht kennen.
- Rest-Elemente können an jeder beliebigen Stelle in einem Tupel vorkommen.

Beispiel:

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

Boxed Types

Boxed Types bezeichnen Wrapper-Objekte, mit denen primitive Typen als Objekte dargestellt werden. Diese Wrapper-Objekte bieten zusätzliche Funktionen und Methoden, die für die primitiven Werte nicht direkt verfügbar sind.

Wenn Sie auf eine Methode wie `charAt` oder `normalize` eines primitiven `string`-Werts zugreifen, umschließt JavaScript ihn mit einem `String`-Objekt, ruft die Methode auf und verwirft das Objekt anschließend.

Demonstration:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript bildet diese Unterscheidung durch separate Typen für die primitiven Werte und die entsprechenden Objekt-Wrapper ab:

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

Boxed Types werden normalerweise nicht benötigt. Vermeiden Sie Boxed Types und verwenden Sie stattdessen primitive Typen, beispielsweise `string` anstelle von `String`.

Kovarianz und Kontravarianz in TypeScript

Kovarianz und Kontravarianz beschreiben Typbeziehungen innerhalb generischer Typen. Ein `Dog` ist ein `Animal`, aber nicht jedes `Animal` ist ein `Dog`.

Kovarianz (Erzeugen): Eine Funktion, die einen `Dog` zurückgibt, kann eine Funktion ersetzen, die ein `Animal` zurückgibt. Jeder Hund ist ein Tier, aber eine Funktion für beliebige Tiere könnte eine Katze liefern.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;

produceAnimal = produceDog; // Valid
produceDog = produceAnimal; // Error
```

Kontravarianz (Entgegennehmen): Eine Funktion, die jedes `Animal` akzeptiert, kann eine Funktion ersetzen, die einen `Dog` akzeptiert. Sie kann auch Hunde verarbeiten; eine Funktion nur für Hunde kann nicht jedes Tier sicher verarbeiten.

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // Valid
consumeAnimal = consumeDog; // Error with strictFunctionTypes
```

Die Pfeile zeigen die Richtung sicherer Zuweisungen:

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (covariance)
- Consumer<Animal> -> Consumer<Dog> (contravariance)

Veränderbare Arrays: TypeScript erlaubt auch, `Dog[]` an `Animal[]` zuzuweisen. Beide Variablen verweisen auf dasselbe Array; über den allgemeineren Typ kann ein `Animal` eingefügt werden, das kein Hund ist.

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // Allowed, but unsafe
// dogs now contains an Animal that is not a Dog.
```

Compiler-Option: Mit `strictFunctionTypes` werden Parameter eigenständiger Funktionen kontravariant geprüft. Ohne diese Option kann TypeScript beide Richtungen zulassen (Bivarianz), auch wenn dies unsicher ist. Parameter in Methoden- und Konstruktordeklarationen bilden eine Ausnahme.

Optionale Varianzannotationen für Typparameter

Seit TypeScript 4.7 kennzeichnet `out` Kovarianz, `in` Kontravarianz und `in out` Invarianz. TypeScript leitet Varianz normalerweise ab. Annotationen müssen zur tatsächlichen Verwendung des generischen Parameters passen und ändern diese nicht beliebig.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

Template-String-Pattern-Indexsignaturen

Mit Template-String-Pattern-Indexsignaturen können wir flexible Indexsignaturen anhand von Template-String-Mustern definieren. Dieses Feature ermöglicht es uns, Objekte zu

erstellen, die mit bestimmten Mustern von Stringschlüsseln indiziert werden können. Dadurch erhalten wir beim Zugriff auf und bei der Manipulation von Eigenschaften mehr Kontrolle und Spezifität.

Seit Version 4.4 unterstützt TypeScript Indexsignaturen für Symbole und Template-String-Muster.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
    [key: MyKeys]: number;
};

const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
    'key-a': 123,
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Unique symbol key
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

Der satisfies-Operator

Mit dem Operator `satisfies` können Sie prüfen, ob ein bestimmter Typ einem bestimmten Interface oder einer bestimmten Bedingung entspricht. Mit anderen Worten stellt er sicher, dass ein Typ alle erforderlichen Eigenschaften und Methoden eines bestimmten Interfaces besitzt. So kann

sichergestellt werden, dass eine Variable zu einer Typdefinition passt. Hier ist ein Beispiel:

```
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Type Annotation using `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// In the following lines, TypeScript won't be able to infer properly
user.attributes?.map(console.log); // Property 'map' does not exist on type 'string | string[]'. Property 'map' does not exist on type 'string'.
user.nickName; // string | string[] | undefined

// Type assertion using `as`
const user2 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// Here too, TypeScript won't be able to infer properly
user2.attributes?.map(console.log); // Property 'map' does not exist on type 'string | string[]'. Property 'map' does not exist on type 'string'.
user2.nickName; // string | string[] | undefined

// Using the `satisfies` operator we can properly infer the types now
const user3 = {
    name: 'Simon',
    nickName: undefined,
```

```
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log);    // TypeScript infers
correctly: string[]
user3.nickName; // TypeScript infers correctly: undefined
```

Type-Only Imports und Export

Mit Type-Only Imports und Exports können Sie Typen importieren oder exportieren, ohne die mit diesen Typen verbundenen Werte oder Funktionen zu importieren beziehungsweise zu exportieren. Dies kann dazu beitragen, die Größe Ihres Bundles zu reduzieren.

Für Type-Only Imports können Sie das Schlüsselwort `import type` verwenden.

TypeScript erlaubt in Type-Only Imports sowohl Dateierweiterungen für Deklarations- als auch für Implementierungsdateien (.ts, .mts, .cts und .tsx), unabhängig von den Einstellungen für `allowImportingTsExtensions`.

Zum Beispiel:

```
import type { House } from './house.ts';
```

Die folgenden Formen werden unterstützt:

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

using-Deklaration und Explicit Resource Management

Eine `using`-Deklaration ist eine unveränderliche Bindung mit Blockgültigkeitsbereich, ähnlich wie `const`, die zum Verwalten freizugebender Ressourcen verwendet wird. Bei der Initialisierung mit einem Wert wird die Methode `Symbol.dispose` dieses Werts registriert und anschließend beim Verlassen des umgebenden Blockgültigkeitsbereichs ausgeführt.

Dies basiert auf dem Resource-Management-Feature von ECMAScript, das zur Durchführung wichtiger Bereinigungsaufgaben nach der Objekterstellung dient, beispielsweise zum Schließen von Verbindungen, Löschen von Dateien und Freigeben von Speicher.

Hinweise:

- Da diese Funktion erst kürzlich mit TypeScript-Version 5.2 eingeführt wurde, unterstützen die meisten Laufzeitumgebungen sie nicht nativ. Sie benötigen Polyfills für: `Symbol.dispose`, `Symbol.asyncDispose`, `DisposableStack`, `AsyncDisposableStack`, `SuppressedError`.
- Darüber hinaus müssen Sie Ihre tsconfig.json wie folgt konfigurieren:

```
{
    "compilerOptions": {
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

Beispiel:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Simple polyfill

const doWork = (): Disposable => {
    return {
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // Resource is declared
    console.log(2);
} // Resource is disposed (e.g., `work[Symbol.dispose]()` is
evaluated)

console.log(3);
```

Der Code gibt Folgendes aus:

```
1
2
disposed
3
```

Eine Ressource, die freigegeben werden kann, muss dem Interface `Disposable` entsprechen:

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

Die `using`-Deklarationen speichern die Operationen zur Ressourcenfreigabe in einem Stack, sodass die Ressourcen in

umgekehrter Reihenfolge ihrer Deklaration freigegeben werden:

```
{
    using j = getA(),
        y = getB();
    using k = getC();
} // disposes `C`, then `B`, then `A`.
```

Die Freigabe von Ressourcen ist auch dann garantiert, wenn nachfolgender Code ausgeführt wird oder Ausnahmen auftreten. Dies kann dazu führen, dass die Freigabe selbst eine Ausnahme auslöst und dabei möglicherweise eine andere unterdrückt. Um Informationen zu unterdrückten Fehlern zu erhalten, wird die neue native Ausnahme `SuppressedError` eingeführt.

await using-Deklaration

Eine `await using`-Deklaration verarbeitet eine asynchron freizugebende Ressource. Der Wert muss über eine Methode `Symbol.asyncDispose` verfügen, deren Abschluss am Ende des Blocks abgewartet wird.

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // Resource is declared
}     //     Resource     is     disposed     (e.g.,     `await
work[Symbol.asyncDispose]()` is evaluated)
```

Eine asynchron freizugebende Ressource muss entweder dem Interface `Disposable` oder `AsyncDisposable` entsprechen:

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
```

```
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose ??= Symbol('Symbol.asyncDispose'); // Simple polyfill

class DatabaseConnection implements AsyncDisposable {
    // A method that is called when the object is disposed asynchronously
    [Symbol.asyncDispose]() {
        // Close the connection and return a promise
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
        await new Promise(resolve => setTimeout(resolve, 1000));
        console.log('Connection closed.');
    }
}

async function doWork() {
    // Create a new connection and dispose it asynchronously when it goes out of scope
    await using connection = new DatabaseConnection(); // Resource is declared
    console.log('Doing some work...');
}    // Resource is disposed (e.g., `await connection[Symbol.asyncDispose]()` is evaluated)

doWork();
```

Der Code gibt Folgendes aus:

```
Doing some work...
Closing the connection...
Connection closed.
```

Die Deklarationen `using` und `await using` sind in folgenden Anweisungen zulässig: `for`, `for-in`, `for-of`, `for-await-of`, `switch`.

Importattribute

Die Importattribute von TypeScript 5.3 (Kennzeichnungen für Imports) teilen der Laufzeitumgebung mit, wie Module (JSON usw.) behandelt werden sollen. Dies verbessert die Sicherheit durch eindeutige Imports und ist auf die Content Security Policy (CSP) abgestimmt, um Ressourcen sicherer zu laden. TypeScript stellt ihre Gültigkeit sicher, überlässt jedoch der Laufzeitumgebung ihre Interpretation für die jeweilige Modulverarbeitung.

Beispiel:

```
import config from './config.json' with { type: 'json' };
```

mit dynamischem Import:

```
const config = import('./config.json', { with: { type: 'json' }
});
```

Syntaxprüfung regulärer Ausdrücke

Seit TypeScript 5.5.4 werden Regex-Literale zur Kompilierzeit auf häufige Fehler geprüft (z. B. ungültige Syntax, falsche Rückverweise oder für Ihre JS-Zielversion nicht unterstützte Funktionen). Dadurch können Fehler früher erkannt werden, Zeichenfolgen in new RegExp(“…”) werden jedoch nicht geprüft.

```
let r = /(a)\2/; // Error: This backreference refers to a group
that does not exist.
```

import defer

Mit `import defer` können Sie ein Modul laden, seine Ausführung jedoch verzögern, bis Sie tatsächlich etwas daraus verwenden. Dadurch lassen sich unnötige Arbeit und Seiteneffekte vermeiden.

- Funktioniert nur mit: `import defer * as name from "module"`
- Der Code wird erst ausgeführt, wenn Sie auf einen Export zugreifen